methane

MDPI

Article

Supervisory Monitoring and Control Using Chemical Process Simulators and SCADA Systems

Rebecca Bastos Boschoski * and Lizandro de Sousa Santos *

Departamento de Engenharia Química e Petróleo, Universidade Federal Fluminense, Rua Passo da Pátria, 156, Niterói 24210240, RJ, Brazil

* Correspondence: rbboschoski@id.uff.br (R.B.B.); lizandrosousa@id.uff.br (L.d.S.S.)

Abstract

A digital twin (DT) is an automation strategy that integrates a physical plant with an adaptive, real-time simulation environment, with bidirectional communication between them. In process engineering, DTs promise real-time monitoring, prediction of future conditions, predictive maintenance, process optimization, and control. Dashboards for process monitoring are becoming increasingly relevant for tracking key metrics and supervising industrial units in real time. Supervisory Control and Data Acquisition (SCADA) systems are widely used for process automation, with ScadaBR, an open-source, freely licensed platform. This work presents the development of a computational tool that integrates the Aspen HYSYS/Python with the ScadaBR system for real-time monitoring and supervision of dynamic models. The virtual plant, which replicates the system's physical behavior, was connected to the SCADA platform via the Modbus protocol, enabling bidirectional data exchange between the simulated model and the supervisory interface. The system supports operational analysis and control strategy validation. Two case studies were analyzed: (i) a simplified catalytic hydrocracking process, implemented in the Python environment, and (ii) a heat exchanger networks process, simulated using the HYSYS simulator. In the second case, the process was dynamically simulated, with real-time monitoring of a simple dynamic indicator that correlates the feed methane concentration with heat transfer fluids. The results demonstrate the feasibility and applicability of the proposed approach for educational purposes, operator training, and process engineering validation, fostering a more realistic and interactive simulation environment. Furthermore, the results show that the tool is promising for dynamic monitoring of environmental and energy indices, demonstrating that methane consumption relative to process feed can be evaluated and controlled over time.

Keywords: chemical process; simulation; dynamic model; digital twin

Academic Editor: Patrick Da Costa

Received: 8 October 2025
Revised: 21 January 2026
Accepted: 22 January 2026
Published: 5 February 2026

Copyright: © 2026 by the authors. Submitted for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

In industry, the use of Supervisory Control and Data Acquisition (SCADA) systems is essential [1]. Such systems have relatively high development costs, which makes them unfeasible for small-scale processes. SCADA systems are widely used in industries with complex or continuous operations, such as crude oil refinery plants [2], oil and gas facilities, water treatment [3,4], and large-scale food and beverage processing plants. In these sectors, real-time monitoring and control of variables such as flow, temperature, pressure, and chemical reactions are critical for safe and efficient operation. However, with the

widespread adoption of open-source software, the automation of industrial processes at various scales has become more accessible. The most modern supervisory and control systems encompass not only process visualization but also enable the operator to interact with the system and ongoing processes, making an accessible interface essential for proper process understanding. SCADA systems can achieve this result, ranging from simple applications to full control panels [2].

There is an increasing demand for performance, quality, and flexibility in industrial facilities. Therefore, developing SCADA systems is crucial, as they facilitate a specific interaction between users and the supervised process [3]. In most cases, SCADA solutions provide a Human–Machine Interface (HMI) for control room operations. Control graphic interfaces that represent the plant's status in real time, including icons, animated icons, numbers, graphs, and flows. Additionally, it allows sending commands to the control network to change parameters, turn switches on or off, control valves, and more. [4]. According to Šenk et al. (2024) [5], as the industrial world transitions to more advanced and interconnected infrastructures, the potential benefits of leveraging algorithms for enhanced monitoring, control, and decision-making in SCADA systems become evident, including process safety and cybersecurity [6,7].

According to [2], the inflated cost of SCADA systems makes automation of small projects infeasible. For this reason, there is robust growth in the use of open-source software, which drives its technological development.

Some developers have integrated dynamic models into SCADA systems. In the work of Zou et al. (2003) [8], a dynamic simulator for operator training in small-scale chemical production processes was developed and used to support process operation and optimization. The authors used linear first- and second-order models to represent process dynamics. SCADA Configuration Software, CENTURY STAR, was used to develop the graphical user interface (GUI) of the operator-training simulator. Scholl and Rocha (2016) [9] developed open-source SCADA software (ESSA), designed for small and scientific applications. The tool enables integration with experimental plants, where HMI can be made available directly on the equipment via touch displays or remotely via a mobile/web interface. Rodríguez et al. (2016) [3] developed a software architecture for educational laboratories that utilize industrial virtual plants. Commercial software, such as MATLAB, was used to develop dynamic models. The results demonstrated improved understanding of the relationships among different didactic units and the procedure for developing a SCADA system in an industrial environment. Bagal et al. (2018) [10] developed a real-time control system using a SCADA and MATLAB OPC Toolbox. The design and implementation were carried out in three phases. First, a PLC-based application was designed; subsequently, SCADA applications were developed and integrated with the process via an OPC [11] server. The process control results demonstrated efficient and reliable real-time data exchange among the PLC, MATLAB, and SCADA. Dieu (2001) [12] applied a SCADA system to wastewater treatment facilities. The developed tool could monitor the city's sewage collection and distribution systems, wastewater treatment, and wet-weather facilities. In the work of Morsi et al. (2014) [13], a SCADA/PLC system was utilized to control an entire oil refinery. The design and specific implementation method for a real SCADA/PLC system in an oil refinery process were developed using the RSVIEW-32 SCADA system. In the work of Kovaliuk et al. (2018) [14], an Arduino-based Web SCADA system was implemented to control the absorption process. Corresponding software for the Web SCADA system was also designed. Novák et al. (2014) [15] developed a framework that integrates simulations, data sources, optimizers, other calculations, and SCADA systems into a single environment.

Virtual industrial plants, or digital twins, are essential to help understand theoretical concepts of a real facility, but also to be used as a simulator to replace the real equipment

in the training process [3]. Chemical process simulators, such as Aspen HYSYS [16] and Aspen Plus [17], are also widely used for teaching purposes. They are of great importance to the industry for training and employee qualification. Additionally, there is considerable interest from educational institutions in these tools, both to test theoretically developed behaviors and to assess the need for supervision, control, and instrumentation of the process.

The use of simulators such as Aspen HYSYS [16] and Aspen Plus [17,18] has become increasingly common for process simulation and optimization. In gas processing and petroleum refining, these tools are crucial, primarily because they enable modeling of systems with complex thermodynamic behavior. In the context of natural processing, we can cite the works of [19–22].

Steady-state models provide a "snapshot" of a process in ideal thermodynamic equilibrium. These models are typically used for process synthesis and design, equipment sizing, and determination of energy requirements. Dynamic models, by contrast, account for the accumulation of material and energy. These models capture the transient response of a process and are typically used for process control and design, safety and operability studies, and operator training simulators [23]. Several works have implemented dynamic simulation throughout the use of dynamic simulators: [24–27]. However, the application of real dynamic simulation still encounters modeling and implementation difficulties. Their integration with the SCADA system is not a common task. Table 1 summarizes some open-source SCADA systems.

As shown in Table 1, several open-source SCADA systems are available. We believe that, among these, ScadaBR is sufficient for small-scale implementations and integration with chemical process simulators. Notice that, in addition to the systems listed in Table 1, Church et al. (2017) [28] report other systems, including EPICS, TANGO, and ECLIPSE SCADA.

ScadaBR [29] is an open-source software option for engineering application development. The software enables developers to implement applications with a 100% web interface, providing access to and control over devices and processes via any web browser on a computer, tablet, or smartphone. The main interface is easy to use and offers visualization of variables, graphs, statistics, protocol configuration, alarms, the creation of dashboards, and variable monitoring [29]. Although there are other SCADA tools, such as those listed in Table 1, ScadaBR is not necessarily the best; however, it is suitable for small-scale systems and allows easy communication with Python.

Recently, Miranda and Santos (2025) [30] applied ScadaBR to a gas mixing process with the use of Unisim Design [19] software. However, to date, few studies have been found in the literature that integrate open-source SCADA systems, such as ScadaBR and OpenScada [31] with process simulators (e.g., Aspen HYSYS or Aspen Plus) to establish virtual plants or digital twins featuring real-time dynamic operation [32] validation. This gap highlights an opportunity to develop solutions that integrate computational modeling within dynamic process simulators with SCADA interfaces, thereby expanding the scope of application for these technologies in chemical engineering.

Although this work does not develop novel tools for simulation and monitoring, the main innovation was the integration of dynamic chemical process simulators (Aspen HYSYS) with the ScadaBR system, which has not yet been reported in the literature. However, there are small-scale applications in Brazil that are poorly documented in academic journals. In this context, the present work proposes using the ScadaBR as a graphical user interface for real-time monitoring of chemical process simulations. It represents a promising platform due to its relative ease of connection with other chemical engineering simulators. Two representative case studies are considered: (i) a catalytic hydrocracking

process and (ii) a heat exchanger handling a hydrocarbon stream. So, the main contributions of this work are as follows:

(i) By integrating dynamic models with ScadaBR, this approach aims to provide operator-like interaction and visualization capabilities, thereby bridging the gap between dynamic simulation and practical control-room training.
(ii) Presenting design aspects regarding the implementation of dynamic models with the ScadaBR system, such as time synchronization, data communication, and visual monitoring.
(iii) Discuss new possibilities to implement dynamic performance indicators related to energy usage, environmental factors, and production capacity.

Table 1. Summary of the main open-source SCADA systems.

SCADA System	Customization	Scalability	Integration
ScadaBR [29]	Web environment with a graphical editor and scripting; customizable through API and event/routine configuration. Ideal for visualization adjustments and standardized logic.	Suitable for small to medium-sized projects; performs well in laboratories, small facilities, and IoT.	Supports numerous protocols (Modbus, OPC, DNP3, MQTT, etc.) and APIs, with integration to databases and systems through scripts and APIs.
Rapid SCADA [33] *	Support for modules and plugins; extendable through code and custom module development.	It can be used in large distributed systems, supporting replication, multiple nodes, and integration.	Broad compatibility (Modbus, OPC UA/DA, MQTT).
OpenSCADA [31]	A modular, multi-component architecture enables the addition or replacement of modules (e.g., UI, database, protocols).	Designed to support distributed systems, with modules capable of operating in scalable architectures, including clusters and complex network configurations.	Architecture focused on connecting multiple modules and protocols, facilitating integration with external systems and databases via specific interface modules.
IndigoSCADA [34]	C/C++ code facilitates customization of the SCADA engine itself, drivers, and internal logic.	Suitable for smaller and embedded systems.	Supports various industrial communication protocols and integrated SQL databases; the integration is more technical and depends on the development environment.

* Standard version.

This work is divided into the following topics. In Topic 2, we describe the study's methodology. Topic 3 describes the studied cases. Topic 4 presents the results. The conclusions are presented in Topic 5.

2. Methodology

2.1. Summary of Methodology

The methodology is summarized in the following topics:

(I) Development of process simulation: In this step, a mathematical model is developed using Python or a process simulator.
(II) ScadaBR-Python communication: The Python programming language (version 3.14.2) is used to implement the pyModBusTCP (version 0.3.0) and pymodbus (version 2.5.3) packages, enabling communication between ScadaBR (version 1.0) and Python (version 3.14).

(III) ScadaBR implementation: Each process variable is inserted into the ScadaBR (version 1.0) environment. Also, the dashboard is implemented, containing the process flowsheet of the modeled process and real-time graphics for monitoring the main process variables.

2.2. Development of Process Simulation

The mathematical modeling stage for chemical processes involves applying mass and energy conservation equations, along with constitutive equations. Dynamic modeling of chemical processes can result in lumped-parameter or distributed-parameter systems. Lumped-parameter models usually produce systems of differential and algebraic equations (DAEs) derived from mass and energy balance equations [35]. The following system of equations illustrates the general structure of a dynamic model in chemical engineering.

$$\frac{dm}{dt} = \sum_{i=1}^{M} \dot{m}_i - \sum_{j=1}^{N} \dot{m}_j \quad (1)$$

$$\frac{dn_k}{dt} = \sum_{i=1}^{M} \dot{n}_{i,k} - \sum_{j=1}^{N} \dot{n}_{j,k} + \dot{\Phi}_k \quad (2)$$

$$\frac{dU}{dt} = \sum_{i=1}^{M} \dot{m}_i \hat{h}_i - \sum_{j=1}^{N} \dot{m}_j \hat{h}_j + \dot{Q} + \dot{W} + \dot{\pi} \quad (3)$$

$$\mathbf{f}(\mathbf{x}, \mathbf{p}) = 0 \quad (4)$$

where m is total mass, $\dot{m}$ is mass flow rate, n is mol, $\dot{n}$ is molar flow rate, t is time, U is total internal energy, $\hat{h}$ is specific enthalpy, $\dot{Q}$ is heat rate, $\dot{W}$ is power, $\dot{\pi}$ is generated heat, $\dot{\Phi}$ is molar generation, $\mathbf{f}$ is a general vector or algebraic function, $\mathbf{x}$ is an algebraic variable, and $\mathbf{p}$ is the vector of parameters. In the above equations, k denotes component species, M is the number of input streams, and N is the number of output streams.

These models can be implemented in programming languages such as C++, Python, and MATLAB (MATLAB2025). Chemical process simulators, such as Aveva Dynsim and Aspen Dynamics, can also be used and allow the implementation of more complex models.

In Figure 1, the model includes independent input variables, parameters, and dependent variables. Before integration, the parameters and input variables are updated from ScadaBR. The model is then integrated from time t_0 to $t_0 + \Delta t$. The final values obtained after integration are exported to ScadaBR. This solution scheme operates cyclically.

2.3. ScadaBR–Python Communication

Communication between data from the Aspen HYSYS simulator and Python is already well established in the literature. Details of the communication procedure for Example 2 are provided in Appendix A.

The integration between the dynamic simulation and the SCADA system was implemented using the Modbus TCP protocol, employing the pymodbus library (server) and pymodbusTCP (clients). The implementation was organized into logical-functional stages, as described below.

Step 1:

Each Modbus device contains four classic blocks: coils, discrete inputs, input registers, and holding registers. The StartTcpServer method activates the Modbus TCP server on a local port and keeps the service running continuously. This tool provides a stable environment for external clients to read and write Modbus registers, working as a virtual field device for functional testing and integration.

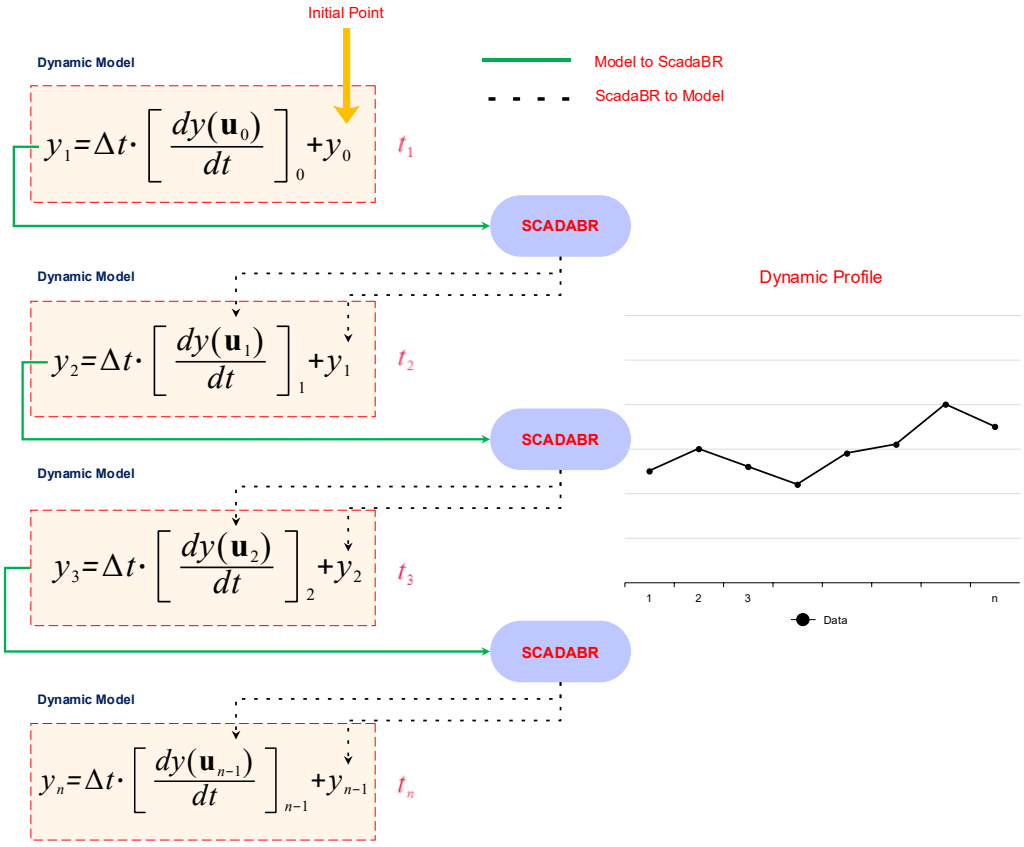

Figure 1. Simplified flowchart of the algorithm used to link ScadaBR and Python (***y***: output variables; **u**: input variables).

Step 2:

Multiple Modbus clients communicate with the Modbus server and the SCADA system using the pymodbusTCP library. Each client is initialized with an IP address. The role of the clients is as follows:

- Open and maintain a Modbus TCP session;
- Read holding registers and input registers;
- Write processed values to Modbus registers.

Step 3:

The Modbus client reads the registers defined as input variables. This reading is performed in blocks, according to the predefined mapping. The values received from the SCADA system are converted to float32 and immediately transferred to the input objects of the dynamic simulator (HYSYS), feeding the corresponding spreadsheets and thermodynamic variables. This step establishes the data flow from the supervisory environment to the dynamic model.

Step 4:

With the input variables updated, the model computes the next process state. Temperatures, pressures, flow rates, compositions, and other thermodynamic and hydraulic variables involved in the process are calculated. After each iteration, the dynamic model

exports the output variables to Python. The time step used for updating the dynamic model was 10 s. This period was defined heuristically to provide a reasonable frequency of process data updates in the ScadaBR system. It is important to emphasize that the computational cost of solving the models, for both examples studied, was well below the update period.

Although PyModbusTCP provides convenient Python-based interfaces for Modbus TCP communication, it can be subject to communication latency and error-handling limitations. A data quality verification routine was implemented at each communication cycle to detect exception errors and communication failures, thereby preventing the transmission of erroneous data to ScadaBR or the computational model. The routine attempts to retrieve data from client devices up to 10 times consecutively before transmission. If data acquisition fails after ten attempts, an alert is automatically issued to notify the user and facilitate the investigation of potential communication issues. During this period, system operation continues using the most recently validated data.

Step 5:

The Modbus client writes these simulated values into the holding registers configured as output points. This step enables the SCADA system to retrieve updated simulation values via the Modbus TCP protocol.

III. ScadaBR Implementation:

Section 3 provides a detailed account of the ScadaBR interface implementation for both case studies. The process generally involves the following steps:

(i) Configuring the DataSource with the Modbus TCP protocol;

(ii) Configuring the DataPoints of each DataSource by linking process variables obtained from the model;

(iii) Configuring the ScadaBR Watchlist, which serves as the main monitoring dashboard displaying the variables;

(iv) Configuring the graphical process interface, including process flow diagrams (PFDs);

(v) Developing real-time monitoring graphs.

Figure 2 summarizes the algorithm.

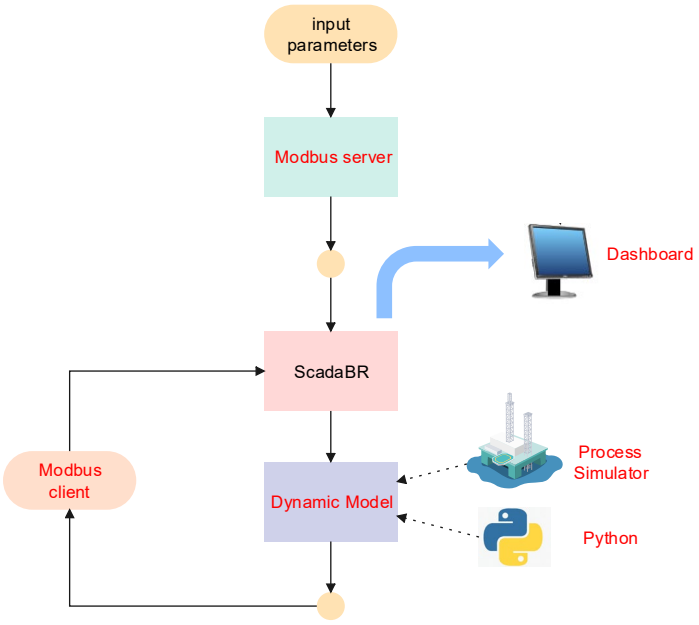

Figure 2. Simplified flowchart of the algorithm used to link ScadaBR and Python.

(1) The Modbus server must be continuously running prior to system initialization.
(2) All variables (datapoints) are initialized within the ScadaBR environment.
(3) The dynamic model is initialized using the initial values provided by ScadaBR.
(4) After the model is integrated, the output variables are exported to ScadaBR via a Modbus client, where the values of input variables can be modified before the initialization of a new cycle. A support Python tutorial for ScadaBR—Python communication can be seen in https://github.com/LizandroCloud/SCADABR-PYTHON (accessed on 21 January 2026), in which case-2 files (Python-HYSYS) are shared.

3. Studied Cases

3.1. Case 1: Kinetic Hydrocracking Model

Hydrocracking has contributed significantly to petroleum refining by converting heavy petroleum fractions into higher-value feedstocks, which are lighter and more valuable as fuels. Hydrocracking commonly consists of two stages. The first stage breaks down sulfur and nitrogen compounds and hydrogenated aromatics. The liquid fraction from the first stage is hydroisomerized and hydrocracked in the second stage [36,37].

Hydrogen is injected into the hydrocracking system at a lower temperature than the operating mixture to reduce the system temperature, since the reactions are exothermic. Hydrogen is also required to form smaller molecules. Figure 3 shows a process flow diagram of a simplified hydrocracking unit. In this process, a mixture of recycled hydrogen, hydrogen (quench), and feedstock is heated through preheating with the reactor effluent, followed by heating in a furnace, before being sent to the reaction section [38].

Hydrocracking reactors contain several catalyst beds separated by sections equipped with feed redistribution systems and hydrogen injections to cool the reaction zone. Temperature control in the reactor helps reduce catalyst deactivation and coke formation rates. When the production objective is to maximize naphtha, five or six beds are typically employed, with four or five intermediate hydrogen injections [38].

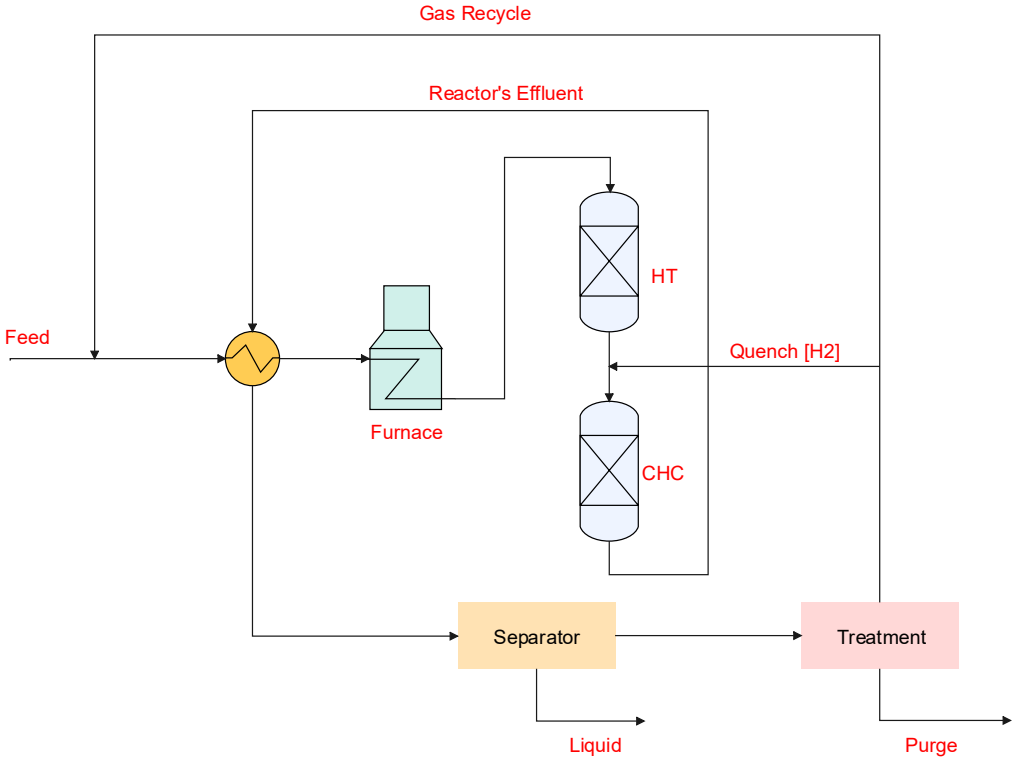

Figure 3. Hydrocracking block diagram [38] (HT: hydrotreating; CHC: catalytic hydrocracking).

Although other more detailed and complex approaches have been developed the complexity of petroleum feeds suggest that models based on lumping theory can reasonably represent the kinetics of hydrocracking of heavy oi [39].

In this case, a simple network of four parallel reactions was introduced for the main products of the hydrocracking process of a heavy hydrocarbon stream: liquefied petroleum gas (LPG), naphtha, kerosene, and diesel. The reaction rates are described by the Equations (5)–(9):

$$\mathrm{r_{feed}} = -(\mathrm{K_1} + \mathrm{K_2} + \mathrm{K_3} + \mathrm{K_4}) \cdot \mathrm{y_{feed}} \quad (5)$$

$$\mathrm{r_{diesel}} = \mathrm{K_1 y_{feed}} = \left(\mathrm{K_{01}} \cdot e^{(-\mathrm{E_1/RT})}\right) \cdot \mathrm{y_{feed}} \quad (6)$$

$$\mathrm{r_{kerosene}} = \mathrm{K_2 y_{feed}} = \left(\mathrm{K_{02}} e^{(-\mathrm{E_2/RT})}\right) \cdot \mathrm{y_{feed}} \quad (7)$$

$$\mathrm{r_{naphta}} = \mathrm{K_3 y_{feed}} = \left(\mathrm{K_{03}} e^{(-\mathrm{E_3}/RT)}\right) \cdot \mathrm{y_{feed}} \quad (8)$$

$$\mathrm{r_{LPG}} = \mathrm{K_4 y_{feed}} = \left(\mathrm{K_{04}} e^{(-\mathrm{E_4/RT})}\right) \cdot \mathrm{y_{feed}} \quad (9)$$

in which y_{feed} is vacuum gas oil mass fraction, r is the reaction rate, K_i, $i = 1, \cdots, 4$ are kinetic parameters, E_i, $i = 1, \cdots, 4$ are activation rates, K_{0i}, $i = 1, \cdots, 4$, are pre-exponential rates and T is the temperature. The mass balance of each pseudo-component in the reactor is expressed as follows:

$$\frac{dy_i}{d\tau} = \pm r_i \quad (10)$$

where i is any product (naphtha, kerosene, LPG, or diesel) or vacuum gas oil in the mixture, τ is the residence time, and r_i is the rate equation for reactant consumption or product formation involved in the model.

The system of ordinary differential equations (ODEs), as described by Equation (10), was solved using Python's odeint/sympy toolbox. The initial mass fractions of the components were defined as follows: vacuum gas oil VGO (90%), diesel (10%), kerosene (0%), naphtha (0%), and LPG (0%). The adopted feed mass fractions, based on [37] are consistent with typical hydrocracking feedstocks, which are predominantly composed of VGO, with possible contributions from heavier diesel-range material, while lighter fractions are mainly formed during the process. The residence time of the species, $\tau \in [0, 2]$ [h], was used as the independent variable to track the evolution of the mass fractions throughout the reactor.

3.2. Case 2: Heat Exchange Network

This example is based on the work of [30], which implemented a simple heat exchange process for a hydrocarbon stream, based on the study by [40]. In previous work, Siqueira & Santos [20] implemented this process in the HYSYS simulator, but considering steady-state conditions. In the present work, we incorporate the dynamic simulation of this system, as explained below, including the addition of proportional-integral-derivative (PID) control loops.

The PFD (process flow diagram) of the simulated process, proposed by [40], is presented in Figure 4. In the present process, the goal is to cool a mixture of hydrocarbons with a temperature of −6 °C, pressure of 68.95 bar, and molar flow of 1150 kgmol/h. The component molar fractions of the input stream are listed in Table 2.

Table 2. Molar fraction of the feed stream.

Substance	Molar Fraction
Methane (CH_4)	0.7515
Ethane (C_2H_6)	0.2004
Propane (C_3H_8)	0.0401
i-butane (i-C_4H_{10})	0.0040
n-butane (n-C_4H_{10})	0.0040

According to Figure 4, the input stream 1B is divided into two streams. Stream 1A has 923 kmol/h, 68.95 bar, and 6.7 °C., and stream 4 has 227 kmol/h. Stream 3 passes through the heat exchangers E-101 and E-102, where it is cooled to −37.5 °C. Stream 2 passes through the heat exchanger (E-100) to be cooled to −20 °C. The Peng–Robinson [41] thermodynamic model was used. Detailed data can be checked in [30].

Three PID controllers were added to the system. The flow controller FIC-01 was used to regulate the unit's feeding flow rate. The temperature controllers TIC-01 and TIC-02 were used to control the temperatures of streams 6 and 7, respectively. The control-loop parameters are shown in Table 3.

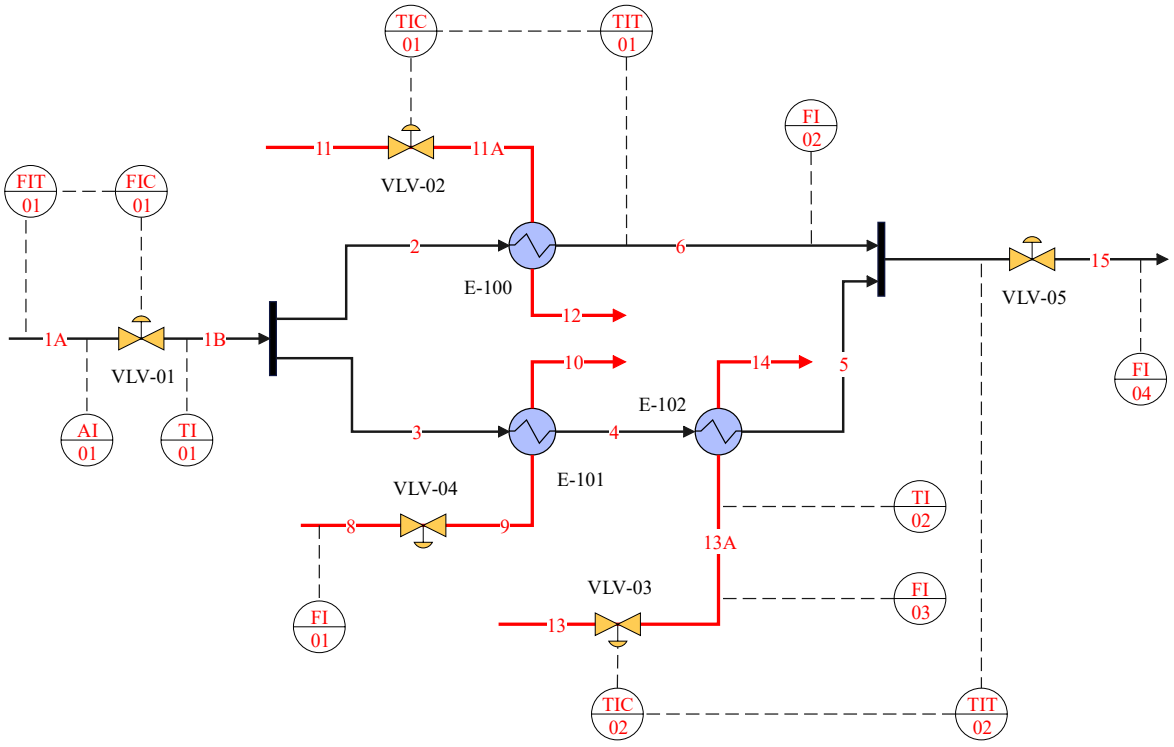

Figure 4. HYSYS flowsheet of dynamic simulation applied to a heat exchanger network (dashed line corresponds to signal transmission and continuous line corresponds to process flows).

Table 3. Configuration of PID controllers.

PID Controller	K_C	τ_i [s]	τ_D [s]
FIC-01	1.33	0037	-
TIC-01	5.87	0.55	0.0397
TIC-02	12.9	0.053	0.0190

Five automatic control valves: VLV-01, VLV-02, VLV-03, and two manual control valves: VLV-04 and VLV-05 are also shown in Figure 4. Implementation details in ScadaBR are presented in Appendix A.

In Figure 4, we observed two temperature-control systems that manipulate the utility flow rates (streams 11 and 13) for heat exchange in E-100 and E-102 heaters. As the temperatures of streams "7" and "6" vary due to possible disturbance in the process, TIC-01 and TIC-02 controllers manipulate the utility flow rates, streams "11" and "13", respectively. A performance index that relates utility consumption to the methane mass fraction in the feed is defined according to Equation (11):

$$\mathrm{KP_{CH4,i}} = \frac{(\dot{m}_i - \dot{m}_{i,min})/(\dot{m}_{i,max} - \dot{m}_{i,min})}{y_{CH4}} \quad (11)$$

where $\mathbf{KP_{CH4,i}} \in [\mathbf{0}, \mathbf{1}]$ is the utility consumption index in relation to the molar fraction of methane in the feed stream, $\dot{\boldsymbol{m}}_{\boldsymbol{i}}$ is the $\boldsymbol{i_{TH}}$ molar flow rate of a cold utility, and $\boldsymbol{y_{CH4}}$ is the molar fraction of methane in the feed stream (1A). The bounds $\dot{\boldsymbol{m}}_{\boldsymbol{i,min}}$ and $\dot{\boldsymbol{m}}_{\boldsymbol{i,max}}$ were estimated by numerical tests in the simulation and are given in Table 4.

Table 4. Minimum and maximum flow rates, in kgmol/h, for utility streams.

Utility Stream	$\dot{m}_{i,min}$	$\dot{m}_{i,max}$
11	0	655.2
13	0	316.2

The block diagram representing the PID system is shown in Figure 5:

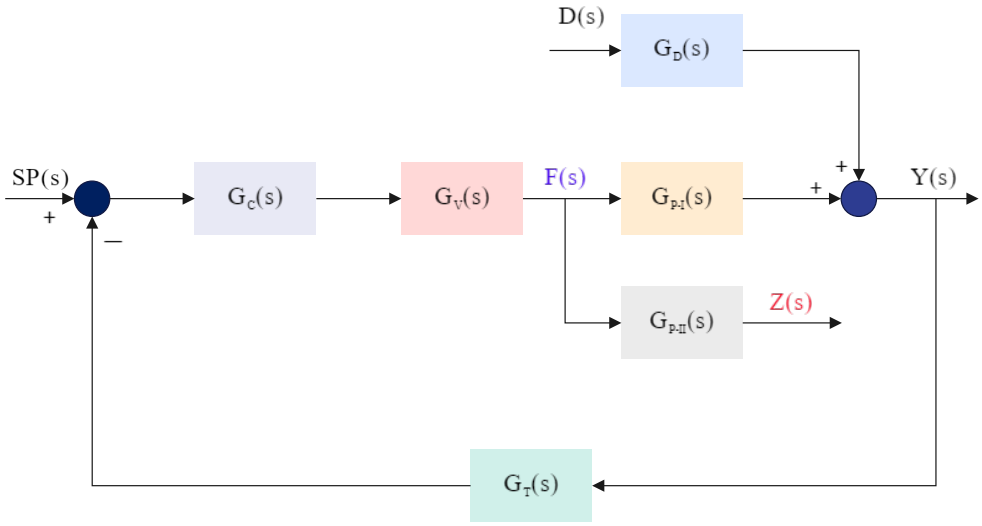

Figure 5. Representation of a feedback PID control block.

In Figure 5, $\mathbf{SP(s)}$ is the set-point, $\mathbf{F(s)}$ is the manipulated variable, $\mathbf{D(s)}$ is the disturbance variable, $\mathbf{Y(s)}$ is the control variable and $\mathbf{Z(s)}$ is the performance index (Equation (7)), in which $\mathbf{s}$ is the Laplace variable. In addition, $\mathbf{G_C(s)}$, $\mathbf{G_V(s)}$, $\mathbf{G_T(s)}$, $\mathbf{G_D(s)}$, and $\mathbf{G_{P-I}(s)}$ and $\mathbf{G_{P-II}(s)}$ denotes the transfer functions of PID controller, valve, transmission, disturbance and processes, respectively [42].

The closed-loop transfer functions can be represented by Equations (12) and (13):

$$\mathbf{Y(s)} = \left[\frac{\mathbf{G_C(s)G_V(s)G_{P-I}(s)}}{\mathbf{1 + G_T(s)G_C(s)G_V(s)G_{P-I}(s)}}\right]\mathbf{SP(s)} + \left[\frac{\mathbf{G_D(s)}}{\mathbf{1 + G_T(s)G_C(s)G_V(s)G_{P-I}(s)}}\right]\mathbf{D(s)} \quad (12)$$

$$\mathbf{Z(s)} = \left[\frac{\mathbf{G_C(s)G_V(s)G_{P-II}(s)}}{\mathbf{1 + G_T(s)G_C(s)G_V(s)G_{P-I}(s)}}\right]\mathbf{SP(s)} - \left[\frac{\mathbf{G_C(s)G_V(s)G_T(s)G_{P-II}(s)G_D(s)}}{\mathbf{1 + G_T(s)G_C(s)G_V(s)G_{P-I}(s)}}\right]\mathbf{D(s)} \quad (13)$$

In the process considered, the disturbance variable $\mathbf{D(s)}$ is the methane fraction in the feed, $\boldsymbol{y_{CH4}}$, whose relationship with the performance index $\mathbf{KP_{CH4,i}}$ depends on the transfer function represented by Equation (9). Also, the controlled variable $\mathbf{Y(s)}$ is therefore influenced by both the disturbance and the setpoint.

The systems represented by Equations (12) and (13) can be simplified according to Equation (14):

$$\begin{bmatrix}\mathbf{Y(s)}\\ \mathbf{Z(s)}\end{bmatrix} = \begin{bmatrix}\mathbf{G_1(s)} & \mathbf{G_2(s)}\\ \mathbf{G_3(s)} & \mathbf{-G_4(s)}\end{bmatrix}\begin{bmatrix}\mathbf{SP(s)}\\ \mathbf{D(s)}\end{bmatrix} \quad (14)$$

in which $\mathbf{G_i, i = 1, 2, 3, 4}$ are the closed loop transfer functions.

4. Results

4.1. Results of Case 1: Kinetic Hydrocracking Model

The Nelder–Mead [43] algorithm was used to estimate the kinetic model parameters based on the model of Barkhordari et al. (2009) [37]. For Nelder–Mead, the minimize function from the Python library scipy was utilized. The following parameters were set: reflection coefficient: 1.0, expansion coefficient: 2.0, contraction coefficient: 0.5 and shrink coefficient: 0.5. The calculated kinetic constant values are listed in Table 5.

The catalytic hydrocracking process was performed in an isothermal packed-bed tubular reactor under different operating conditions, such as reaction temperatures of 390 °C and 410 °C (according to Barkhordari's work [37]), since the kinetic parameters depend on temperature, as shown by the Arrhenius equation.

Table 5. Kinetic parameters.

Parameter	Value	Unit
K_{01}	9.33×10^7	1/h
K_{02}	1.64×10^7	
K_{03}	1.47×10^{13}	
K_{04}	3.61×10^7	
E_1	1.23×10^5	J/mol
E_2	1.01×10^5	
E_3	1.78×10^5	
E_4	1.12×10^5	

Figures 6 and 7 show typical mass–fraction profiles for VGO and its products at 410 °C and 390 °C, respectively. It indicates that the model reasonably reproduces the lower-temperature behavior. At 410 °C, the discrepancies are most pronounced at early times (0.4 h and 0.667 h) and diminish after approximately 1 h. This pattern suggests a small mismatch in the short-term dynamics. By contrast, at 390 °C, the slower kinetics reduce sensitivity to such transients, yielding closer agreement with the experiment over the whole-time horizon.

In hydrocracking, estimating kinetic parameters using the Arrhenius equation often fails to accurately reproduce the concentration profiles due to the process's complexity. The most relevant causes can be explained by the facts: (i) a reaction network more complex than the proposed model; (ii) parameters adjusted for a lump may not correctly represent the internal conversion of the species; (iii) occurrence of temperature gradients in the catalytic bed, and (iv) correlation between the kinetic parameters [36–38]. Deviations

between model predictions and experimental data can also be related to the optimization strategy used for parameter estimation [44].

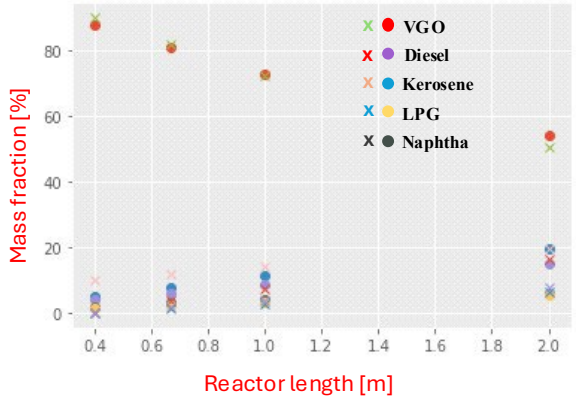

Figure 6. Model result of product's mass fraction percentages along the reactor for $T = 410$ ℃ (circle: reference data; cross: simulated data).

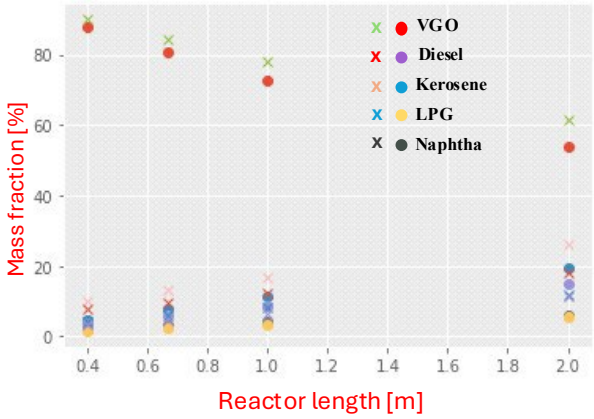

Figure 7. Model result of product's mass fraction percentages along the reactor for $T = 390$ ℃ (circle: reference data; cross: simulated data).

The watchlist displayed in Figure 8 is the easiest way to monitor the values of the imported variables from the virtual CHC reactor. On the left-hand side of the screen, the data points for all variables present in the process are displayed. The right-hand panel allows users to visualize process variables in real time during process simulation. It is also possible to edit the watchlist by moving or removing data points from the main list. In addition, by selecting the "datapoint details" icon, users can visualize statistics, historical data, and graphs related to a specific variable, as shown in Figure 9.

Figure 9 illustrates the typical behavior of a process variable ("Feed") when it is selected in the watchlist. This screen enables users to view statistical data and historical records of the variable over a specified analysis period. The figure illustrates the decrease in the VGO mass fraction from an initial point. As previously presented, the VGO mass fraction decreases along the reactor according to the computed reaction rate.

From the supervisory interface (Figure 10), an operator can adjust the reactor temperature setpoint. On the left side of the screen in Figure 10, it is possible to open the graphs related to the feed variables and the products of the CHC. On the right side of the screen, the representation of the main process can be observed, where the mass percentage values of the products can be visualized in real time.

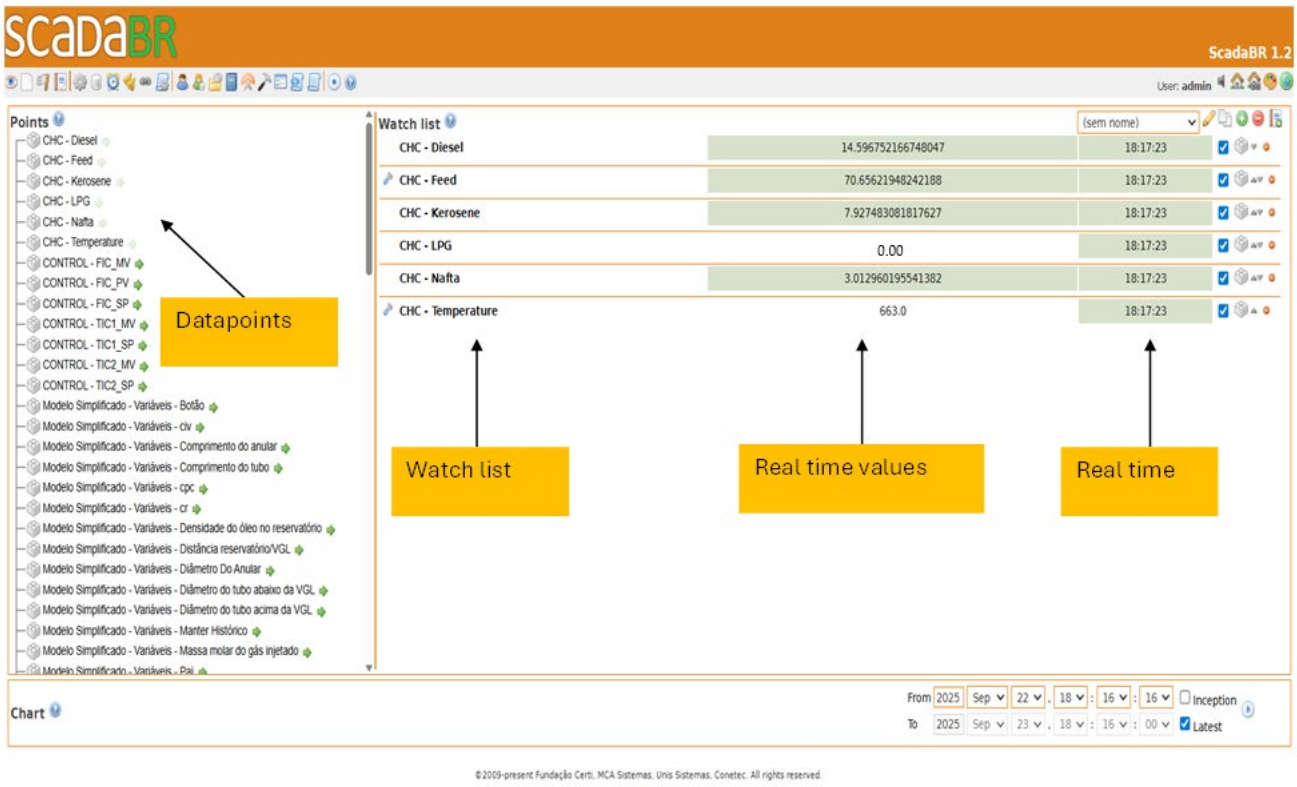

Figure 8. Watchlist of the main variables of the CHC process.

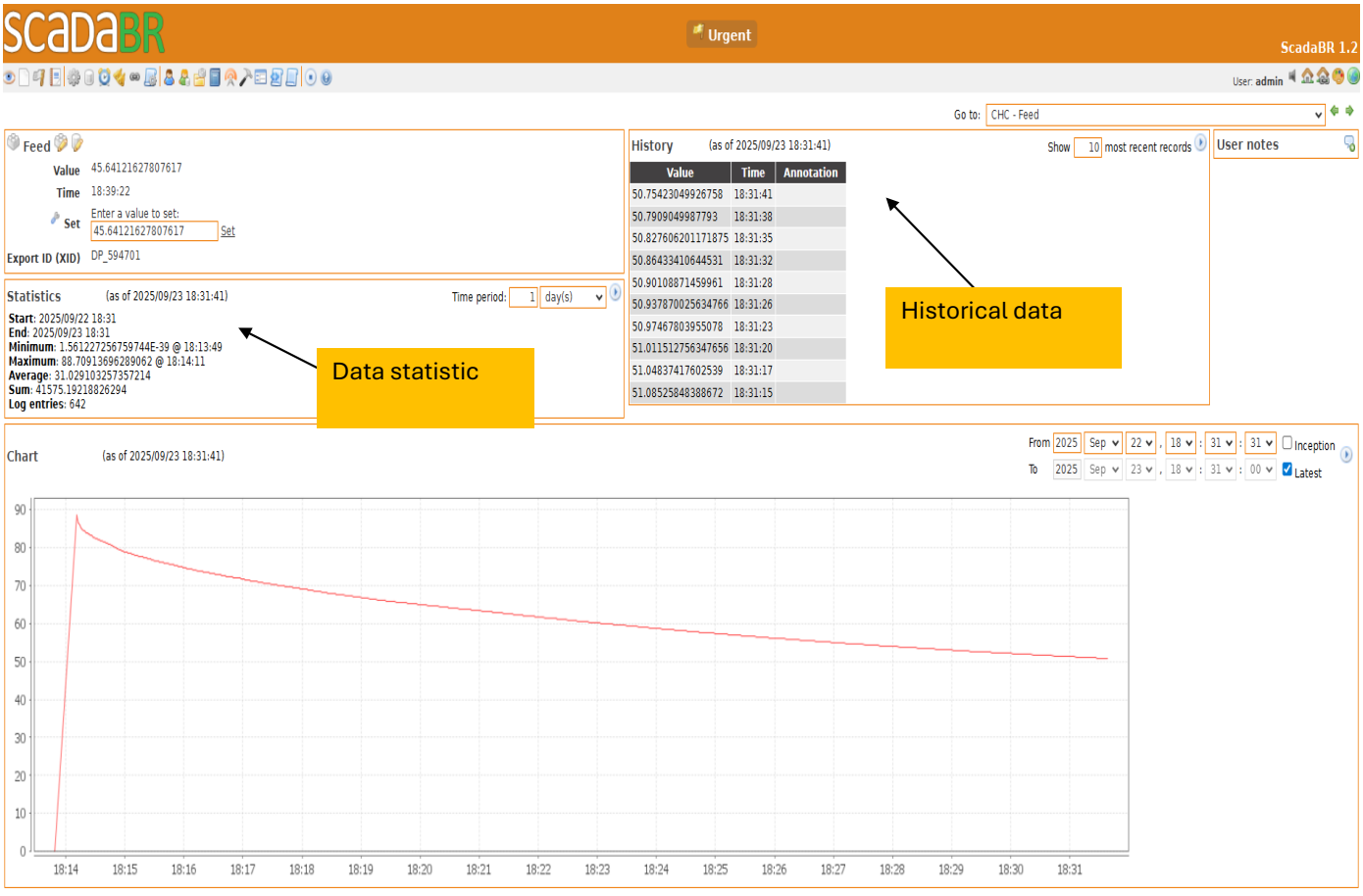

Figure 9. Details of variable "VGO mass fraction".

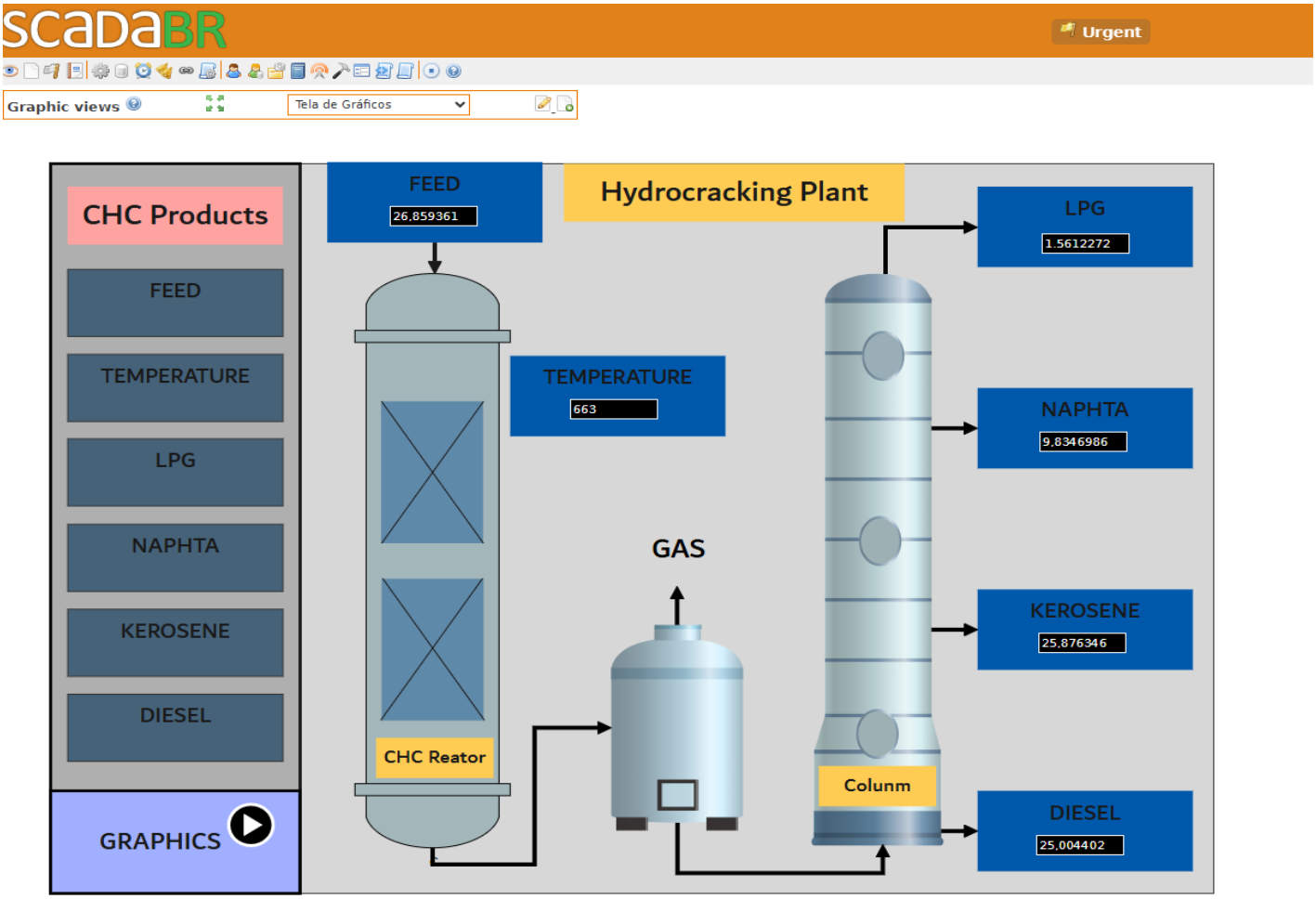

Figure 10. CHC dashboard implemented in ScadaBR.

4.2. Results of Case 2: Heat Exchanger Network

Figure 11 presents the Human–Machine Interface (HMI) of the ScadaBR system used to monitor and control the continuous process under study. The interface provides a high-level plant overview, integrating process equipment, instrumentation, control loops, and operational status indicators within a single supervisory environment. The process is represented by a diagram illustrating the material flow from the feed stream to the product outlet. Pipelines interconnect process units, and the flow direction is explicitly indicated. Real-time measurements of key process variables are displayed numerically, enabling continuous monitoring of plant performance. Temperature and flow variables are measured using temperature transmitters (TT) and flow transmitters (FT), respectively. Analytical measurements are provided by analyzer indicators (AI), which supply real-time information on process composition. This monitoring is supported by an online composition analysis panel, which reports the molar fractions of key components (C1, C2, C3, iC4, and nC4). Control actions are implemented via temperature and flow indicating controllers (TIC and FIC), each associated with a defined setpoint (SP). Several closed-loop control systems are implemented and visually identified by dashed connections between sensors, controllers, and final control elements. The control loops regulate critical process variables by manipulating control valves (VLV-01, VLV-02, VLV-03), with valve positions expressed as a percentage of the valve's opening. In particular, the feed flow rate is regulated by a flow control loop (FIC-01), whereas the process thermal conditions are maintained by temperature control loops (TIC-01 and TIC-02). As the temperatures of streams 6 and 15 vary, the controllers TIC-01 and TIC-02 manipulate the flow rates of streams 11 and 13, respectively. The process also includes multiple heat exchangers (E-100, E-101,

and E-102) that enable energy integration and temperature control across process streams. Operational status is communicated through visual indicators that differentiate normal and critical operating conditions. This alarm visualization strategy enhances situational awareness and supports timely operator intervention during process disturbances.

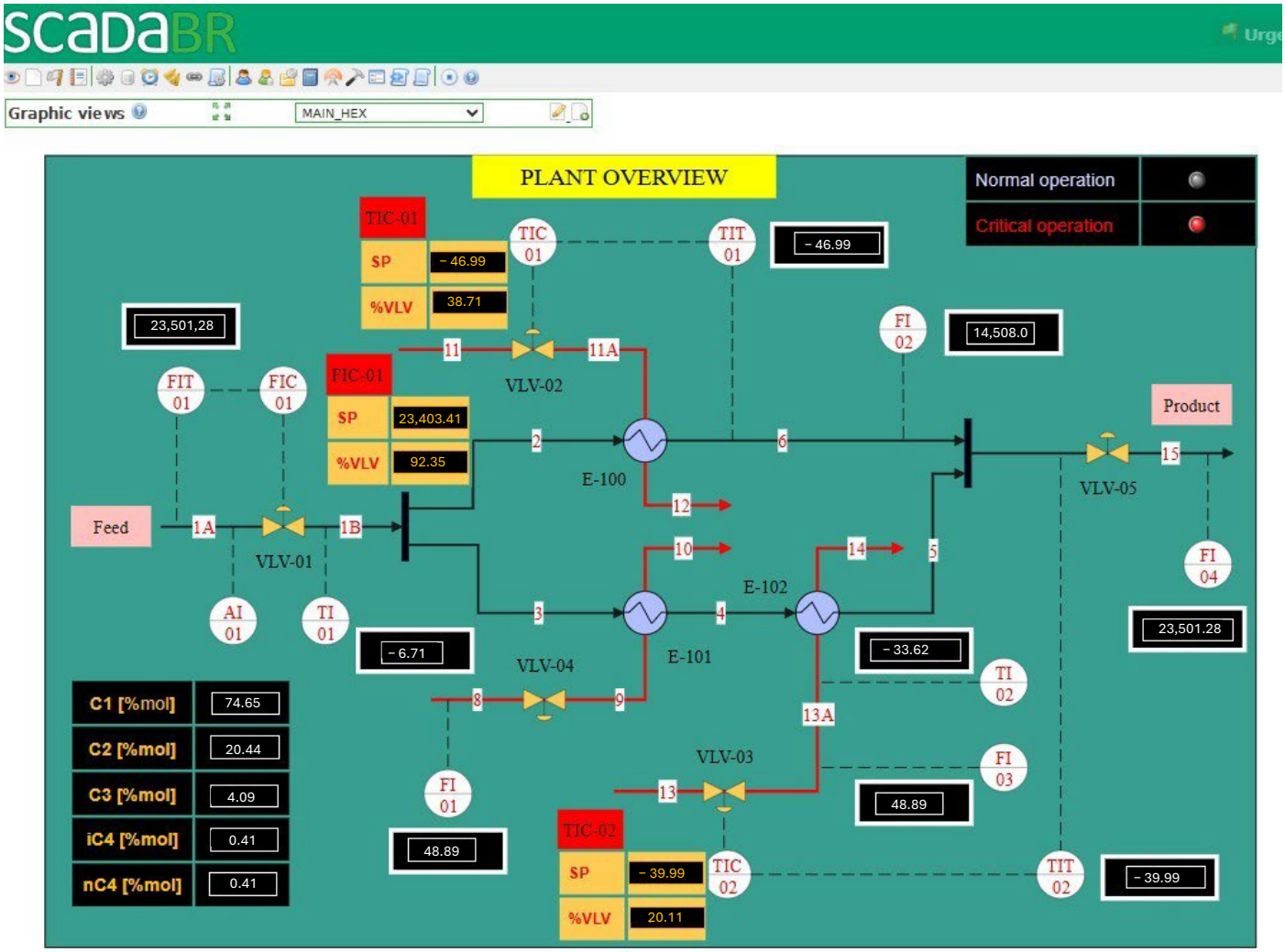

Figure 11. Heat exchanger network process dashboard implemented in ScadaBR.

Figure 12 illustrates the supervisory interface designed for monitoring and performance analysis of closed-loop control systems. The control loops screen consolidates essential information required to evaluate controller behavior under dynamic operating conditions.

The interface displays multiple control loops concurrently, including a flow control loop (FIC-01) and two temperature control loops (TIC-01 and TIC-02). For each loop, time-domain plots of the process variable (PV) and the corresponding setpoint (SP) are presented, enabling direct assessment of setpoint tracking performance and disturbance rejection.

The displayed trends capture both transient and steady-state behaviors of the controlled variables, enabling identification of key performance characteristics such as response time, overshoot, stability, and steady-state error. Observable step changes in the set point facilitate comparative analysis of controller dynamics across different loops.

Figure 12. Dashboard of PID control loops.

In addition to graphical trends, the interface provides real-time numerical values for the SP, PV, and the manipulated variable, represented as valve opening percentage (%VLV). This data supports a correlation between control actions and process responses, which is essential for controller tuning and diagnostic analysis.

The control-loop performance shown in the previous figure is directly related to the PID controller tuning parameters for each loop: FIC-01, TIC-01, and TIC-02. These parameters include the proportional gain K_c, integral time τ_i, and derivative time τ_D. In Figure 12, the FIC-01-SP and TIC-01-SP were modified at 14h and 14h05min, respectively.

In the FIC-01 (Flow Control Loop), the low proportional gain $K_c = 1.33$ combined with the relatively large integral time $\tau_i = 0.037$, and the absence of derivative action results in a conservative controller. This leads to a smooth, stable response with minimal oscillations but a longer setting time.

In the TIC-01 (Temperature Control Loop 1), a moderate to high proportional gain $K_c = 5.87$, moderate integral time $\tau_i = 0.55$ s and a small but effective derivative time $\tau_D = 0.0397$ s, this loop exhibits a faster response with improved damping. The derivative action anticipates the rate of change of the error, reducing overshoot and helping stabilize the control action. As a result, the temperature quickly approaches the set point with minimal overshoot, as evidenced by the corresponding time series.

In TIC-02 (Temperature Control Loop 2), the high proportional gain $K_c = 12.9$, combined with a small integral time $\tau_i = 0.053$ s, and a small derivative time $\tau_D = 0.0190$ s, leads to an aggressive controller. The fast integral action rapidly eliminates steady-state error but can cause overshoot and oscillations due to increased controller sensitivity and the risk of integral windup. The corresponding graph confirms the rapid response with noticeable transient oscillations and overshoot before settling.

Monitoring of the $K_{CH4,i}$ methane indices was also carried out and displayed on the dashboard shown in Figure 13. Two graphs are used to monitor the molar flow rates of the cold utility streams and the molar percentage of methane in the inlet stream (1A). The results show that reducing the molar percentage of methane increases the indices $K_{CH4,1}$ and $K_{CH4,2}$. This can be explained by the fact that the natural gas molar composition directly influences the thermal load required in heat exchangers E-100 and E-102, particularly when phase change occurs, as in the present case. A reduction in the molar fraction of methane, with a consequent increase in the proportion of heavier species (C_2, C_3), raises the dew point temperature of the hydrocarbon mixture. Therefore, in natural gas cooling systems where condensation may occur, a decrease in the methane molar fraction demands more thermal utility to cool the natural gas stream.

Figure 14 shows the correlation between the controlled variables FIC01, TIC01, and TIC02 and the molar fraction of methane at the inlet stream. Changes in the stream's composition result in variations in the properties of the gas stream, including density, viscosity, and average molar mass. These properties directly affect the flow behavior and, consequently, the response of flow measurement and control devices. The results shown in Figure 14 indicate that these disturbances affect the three controlled variables of the process; however, due to the action of the controllers, the variables return to their set-point values.

The results of this example demonstrate that it is feasible to integrate rigorous physical models with the ScadaBR system. This type of application can be helpful for both training and industrial applications. Despite this, the use of ScadaBR systems in large-scale industrial applications has not yet been reported in the literature. For large industrial applications, additional aspects must be addressed, such as: (i) robustness to handle systems with thousands of variables; (ii) cybersecurity; (iii) industrial compliance; and (iv) technical support. Regarding latency, our Modbus TCP integration using Python libraries (PyModbusTCP/pymodbus) demonstrated satisfactory performance for supervisory-level monitoring and set-point tracking in the simulated environment. However, we recognize that in high-speed or highly dynamic processes, the inherent delays in TCP/IP communication, sequential polling, and Python execution may affect real-time performance.

Processing power and large datasets are also relevant factors: the integration was tested on standard desktop hardware and was sufficient for the demonstration cases in HYSYS, but larger simulations with multiple interconnected units or high-frequency data acquisition could require more robust computing resources, such as multi-core processors and increased memory, to maintain responsiveness.

Finally, hardware constraints for real-world deployment should be considered: reliable network infrastructure, appropriate SCADA server capacity, and sufficient client devices are necessary to ensure stable operation. Despite these limitations, the study provides a foundation for future work. It demonstrates that open-source SCADA systems can effectively interface with process simulators, offering a cost-effective and flexible solution for education, research, and small industrial applications.

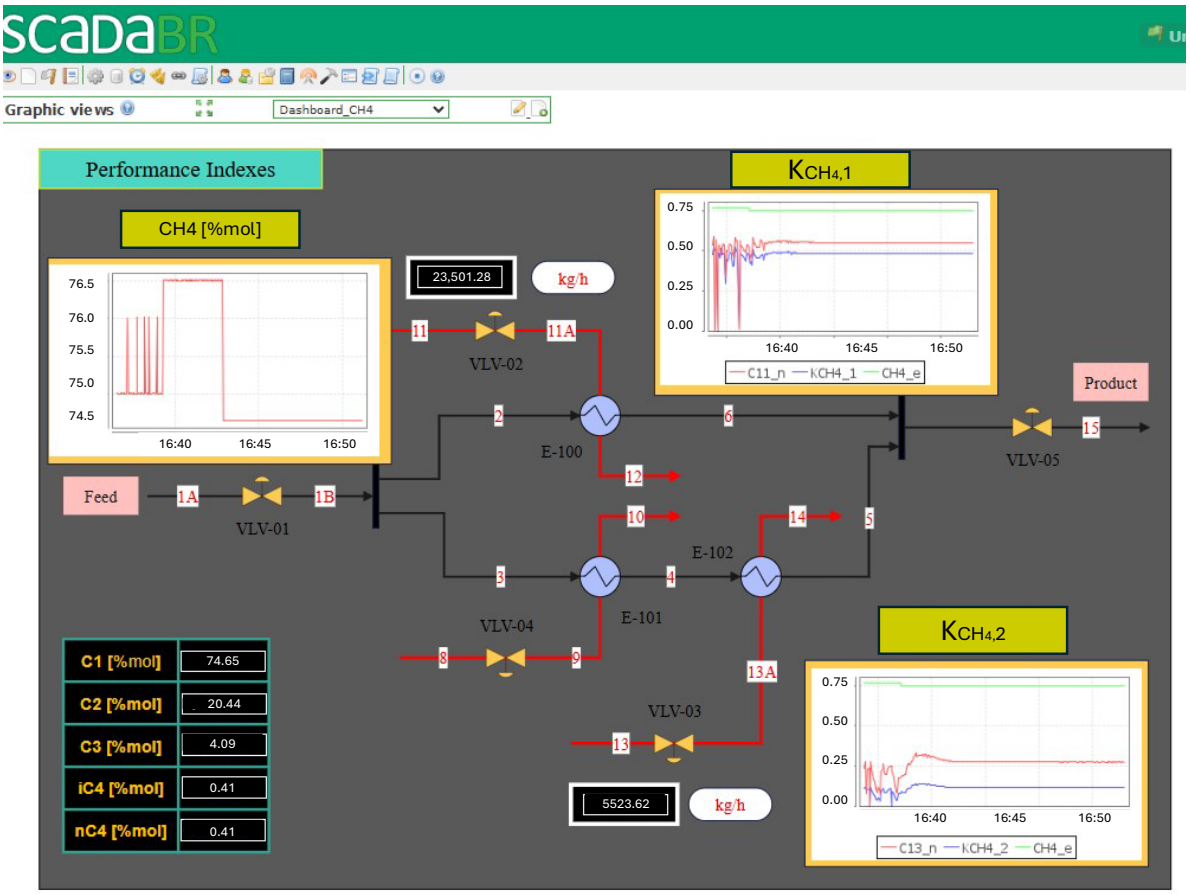

Figure 13. Dashboard of performance indexes.

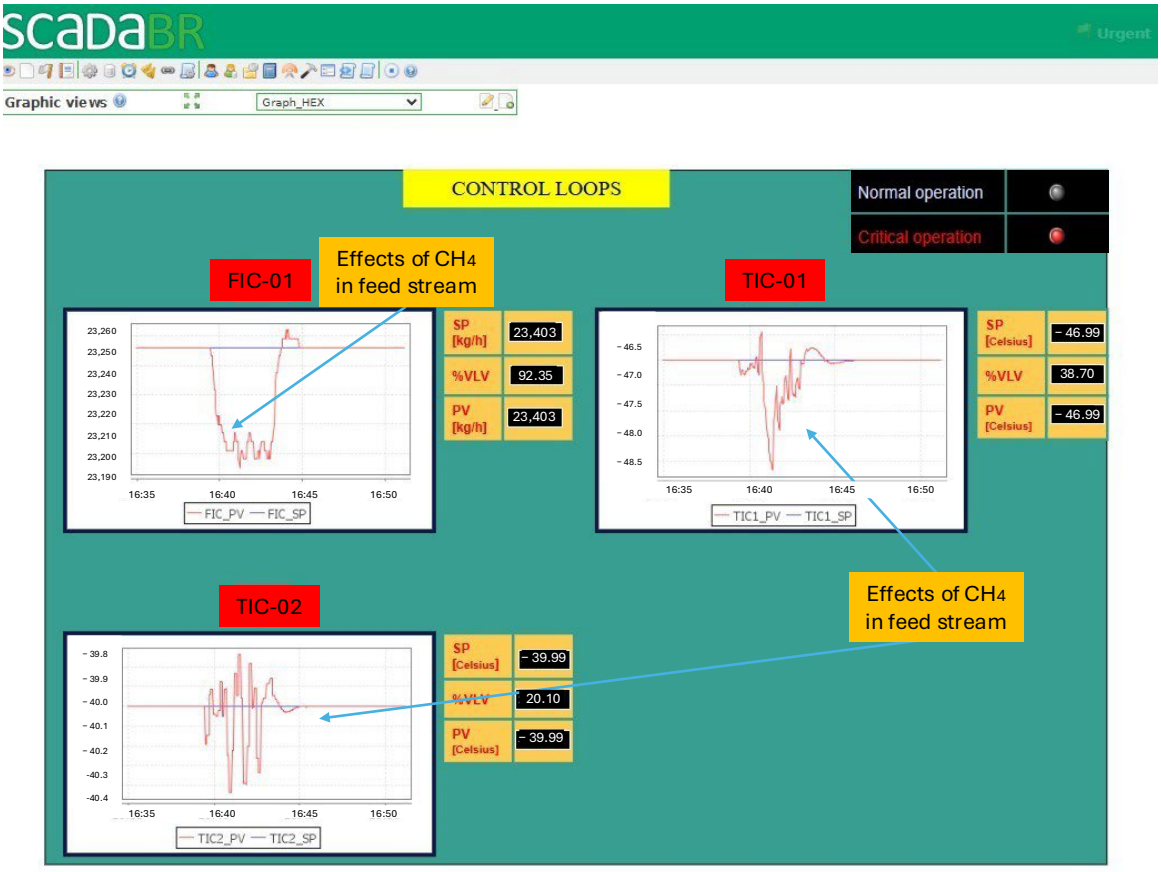

Figure 14. Dashboard for PID control loops (CH_4 effects).

https://doi.org/10.3390/methane5010008

5. Conclusions

This work demonstrates the feasibility of an open-source, web-based supervisory monitoring and control platform that integrates dynamic process simulators with ScadaBR. Two cases were solved.

In the first case, a simplified dynamic kinetic model of a catalytic cracking process of VGO was developed in Python. This model simulates the evolution of the LPG, diesel, naphtha, and kerosene products along the reactor, with periodic updates to the SCADA system. The kinetic model has temperature as the independent variable, which can be adjusted via the ScadaBR dashboard. Tests showed that the ScadaBR system performed well, demonstrating stability and robustness in the communication and representation of its model.

In the second case study, a heat exchanger network simulated in HYSYS Dynamics demonstrated the system's capability to monitor multi-loop PID control, track performance indices under varying feed conditions, and illustrate how changes in methane content affect cooling utility demand.

Indeed, by highlighting the first implementations of dynamic simulation, the work opens the way for future developments. For dynamic simulation, the next step is to integrate dynamic optimization algorithms with predictive control. Another essential point will be the implementation of dynamic machine-learning algorithms (black-box models) that leverage physical models. The tool could also be used to simulate extreme scenarios to predict unsafe process conditions.

Beyond technical validation, the platform offers significant educational and operational advantages. From an educational perspective, the tool offers an interactive digital-twin environment where students and operators can manipulate control variables and setpoints, such as FIC or TIC setpoints, and directly observe the resulting dynamic responses of the process and associated performance indices (e.g., $KPCH_4$). This feature facilitates experiential learning by enabling users to investigate cause-and-effect relationships, control-loop interactions, and transient behaviors that cannot be illustrated in static simulations or steady-state analysis. It improves comprehension of process dynamics, control performance, and operational trade-offs, thereby increasing the platform's value for training and academic instruction.

Future work will focus on expanding the framework to more complex systems (e.g., dynamic distillation columns), incorporating machine learning for predictive analytics, and enhancing the dashboards with features such as mobile notifications and multi-user collaboration.

Author Contributions: Conceptualization, R.B.B. and L.d.S.S.; methodology, R.B.B.; software, R.B.B.; validation, R.B.B. and L.d.S.S.; formal analysis, R.B.B.; investigation, R.B.B.; resources, R.B.B.; data curation, R.B.B. and L.d.S.S.; writing—original draft preparation, R.B.B. and L.d.S.S.; writing—review and editing, R.B.B. and L.d.S.S.; visualization, R.B.B.; supervision, L.d.S.S.; project administration, L.d.S.S.; funding acquisition, L.d.S.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Agência Nacional do Petróleo—ANP, grant number PRH 51.1—Universidade Federal Fluminense—Brazil.

Data Availability Statement: The original data presented in the study are openly available in Github repository at [https://github.com/LizandroCloud/SCADABR-PYTHON], accessed on 7 August 2025.

Acknowledgments: We would like to thank the PRH (Human Resources Program) of the National Agency of Petroleum (ANP) for the financial and structural support provided, which made this

work possible. Finally, I would like to thank the Fluminense Federal University for providing an academic environment conducive to the development of this initiative.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

Symbols	
AI	Analyzer indicator
CHC	Catalytic Hydrocracking
DAE	Differential Algebraic Equations
DCS	Distributed Control System
DT	Digital Twin
E	Exchanger
F	Flow rate
FI	Flow Indicator
FIC	Flow Indicator and Controller
G	Transfer function
J	Joule
LPG	Liquefied Petroleum Gas
P	Pressure
PI	Pressure Indicator
PID	Proportional, Integral, and Derivative
PLC	Programming Logic Controller
PV	Process Variable
SCADA	Supervisory Control and Data Acquisition
T	Temperature
TIC	Temperature Indicator and Controller
TI	Temperature Indicator
VGO	Vacuum Gas Oil
VLV	Valve
Y	Control variable
Z	Process variable
Greek letters	
τ	Residence time
Indices, subscripts, and superscripts	
$(\cdot)_{min}$	Minimum
$(\cdot)_{max}$	Maximum

Appendix A. SCADA-BR Implementation [Case-2]

(I) HYSYS-ScadaBR Communication

Figure A1 illustrates the main steps for implementing communication between ScadaBR and HYSYS.

https://doi.org/10.3390/methane5010008

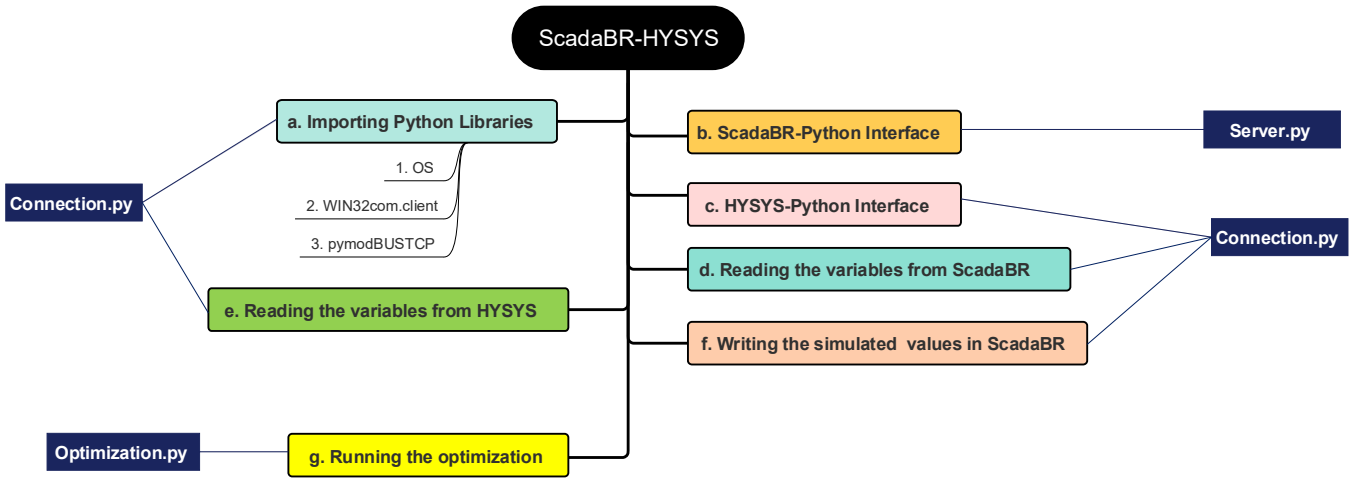

Figure A1. Main steps of the algorithm for communicating ScadaBR and HYSYS.

To connect the ScadaBR with HYSYS, the Python language was used as interface to implement Modbus communication. In the following topics (a–f), the main parts of the "Connection.py" script are explained. A support Python code is available at the GitHub repository (https://github.com/LizandroCloud/SCADABR-PYTHON) (accessed on 21 January 2026).

It is necessary to import the used libraries and establish connections with the Modbus server and HYSYS simulator. In this scheme, Python reads data received from the ScadaBR® through Modbus protocol, reads the information from the simulated plant (implemented into HYSYS), and sends the data back to ScadaBR®. Code 1 lists the main libraries used for the algorithm's implementation.

Code 1: Import Libraries		
	Command	Description
1:	import os	Folder where the files are located
2:	import win32com.client as win32	Link between HYSYS and Python
3:	from pyModbusTCP.client import ModbusClient	Link between Python and ScadaBR
4:	import time	Counting lines

(a) Importing libraries

To communicate the simulation with the Python language, it is necessary to import the used libraries and then establish the connection with the Modbus server and HYSYS simulator. Thus, Python reads data received from the ScadaBR through Modbus, reads the information from the simulated plant, and thus returns computed values to ScadaBR.

(b) Connection to Modbus TCP Server

The use of a Modbus server in a Python code depends on it containing a communication script. For this, the connection script available in PyModbusTCP 0.1.10 ("PyModbusTCP," 2022) was developed as described below:

Code 2: Connecting Modbus with TCP Server	
1:	**class** FloatModbusClient(ModbusClient):
2:	**def** read_float(self, address, number = 1):
3:	reg_l = self.read_holding_registers(address, number × 2)
4:	**if** reg_l:
5:	return [utils.decode_ieee(f) for f in utils.word_list_to_long(reg_l)]
6:	**else**:

```
7:                  return None
8:
9:          def write_float(self, address, floats_list):
10:             b32_l = [utils.encode_ieee(f) for f in floats_list]
11:             b16_l = utils.long_list_to_word(b32_l)
12:         return self.write_multiple_registers(address, b16_l)
```

To use this script, two functions from the pyModbusTCP library are used: "ModbusClient" and "utils". Communication allows writing and reading numerical variables on the servers within a vector, which goes from position 0 to 49. The ports chosen for each server were "5020" and "5022", and to work with the values stored in them, it is necessary to define two objects, defined as "c" and "d", according to the script below:

Code 3: Import Libraries

```
1: c = FloatModbusClient(host = 'localhost', port = 5020, auto_open = True)
2: d = FloatModbusClient(host = 'localhost', port = 5022, auto_open = True)
```

(c) Connecting the HYSYS Design® simulation with Python

To establish the connection between Python and HYSYS, two libraries are needed. The "os" allows the manipulation of operating system files and directories, and the "win32com.client" permits accessing the COM (component object model). The script used to connect the simulated plant with Python is described in Code 4.

Code 4: Python-HYSYS Communication

```
1: unit = win32.Dispatch('HYSYS.Application'')
2: direct = os.path.abspath('.')
3: Case = unit.SimulationCases.Open(direct+'\File_name.hsc')
4: Case.Visible = 1
5: Spread_sheet_1 = Case.Flowsheet.Operations.Item('Simulation')
```

According to Code 4, the imported spreadsheet is saved within the Python environment in the variable "Spread_sheet_1", thus being able to be used within the code. With the connection established, it is possible to read and write all the variables stored in the spreadsheet of HYSYS.

(d) Reading the variables from ScadaBR

The algorithm works in a loop in which the data are read from ScadaBR and HYSYS modified if necessary, and written back to ScadaBR for monitoring. First, the code must read the values coming from ScadaBR through the Modbus TCP server. For this, two variables are defined that store the values of the respective Modbus servers "c" and "d", defined previously. These variables are called "float_1" and "float_2" as shown in the script below:

Code 5: Reading the Variables from ScadaBR

```
1: float_1 = c.read_float(0, 49)
2: float_2 = d.read_float(0, 49)
```

(e) Reading the HYSYS spreadsheet

After reading the variables from ScadaBR, the next step is reading the values of simulated variables from HYSYS for real-time monitoring. Each spreadsheet's cell contains a numerical value that is stored in the "Spread_sheet_1" object as viewed in Code 6.

Code 6: Reading the UniSim Design® Spreadsheet	
1:	S1 = UStr(1)
2:	S1.flow = Spread_sheet_1.Cell('B3').CellValue
3:	S1.ch4 = Spread_sheet_1.Cell('B4').CellValue
4:	S1.c2h6 = Spread_sheet_1.Cell('B5').CellValue
5:	S1.c3h8 = Spread_sheet_1.Cell('B6').CellValue
6:	S1.i_ch4h10 = Spread_sheet_1.Cell('B7').CellValue
7:	S1.n_ch4h10 = Spread_sheet_1.Cell('B8').CellValue
8:	S1.press = Spread_sheet_1.Cell('B9').CellValue
9:	S1.temp = Spread_sheet_1.Cell('B10').CellValue

Notice in Code 6 that the class "UStr " defines the object S1 (stream 1) that inherits several properties: pressure, temperature, flow rate and molar compositions. The name of the variable defined to store the class corresponds to the stream and its number, which in the case described above is "S1". This process is repeated for all 14 streams of the simulated process. The code of the "UStr " class is given in Code 7.

Code 7: Reading the HYSYS Spreadsheet	
1:	**class** UStr:
2:	**def** __init__(self, number):
3:	self.number = number
4:	
5:	**def** props(self, flow, ch4, c2h6, c3h10, i_ch4h10, n_ch4h10, pres, temp):
6:	self.flow = flow
7:	self.ch4 = ch4
8:	self.c2h6 = c2h6
9:	self.c3h10 = c3h10
10:	self.c3h10 = c3h10
11:	self.i_ch4h10 = i_ch4h10
12:	self.n_ch4h10 = n_ch4h10
13:	self.press = pres
14:	self.temp = temp

(f) Writing HYSYS values in ScadaBR

The process variables resulting from the simulation are written in the ScadaBR through the ModbusTCP server. The command "write_float" is used to write each value in the vectors "c" and 'd" defined in Code 3. Code 8 shows the code to write the properties of the 14 streams from the process simulation.

Code 8: Writing HYSYS Values in ScadaBR	
1:	c.write_float(0, [S1.flow, S1.ch4, S1.c2h6, S1.c3h8, S1.i_ch4h10, S1.n_ch4h10, S1.press, S1.temp,S1.flow, S1.ch4, S2.c2h6, S2.c3h8, S2.i_ch4h10, S2.n_ch4h10, S2.press, S2.temp,S3.flow, S3.ch4, S3.c2h6, S3.c3h8, S3.i_ch4h10, S3.n_ch4h10, S3.press, S3.temp,opt])
2:	d.write_float(0, [C4.press, C4.temp, C5.press, C5.temp, C6.press, C6.temp, C8.press, C8.temp, C7.flow, C7.ch4, C7.c2h6, C7.c3h8, C7.i_ch4h10, C7.n_ch4h10, C7.press, C7.temp, C9.press, C9.temp, S10.temp, S11.temp, S11.flow,S12.temp, S12.flow, S13.temp, S13.flow, S14.temp])

Figure A2 summarizes the communication between HYSYS–Python–ScadaBR.

Figure A2 exemplifies the communication of the S1's flow rate to ScadaBR. The value is sent in the B3 cell attributed to the variable "S1.flow". In the sequence, the "S1.flow" is sent to the "c" object connected to the Modbus address 5020.

(II) Development of the supervisory system in ScadaBR

(a) Data Sources and Data Points

The variables connected to the Modbus server are registered as data points in the ScadaBR environment. A set of data points is associated with the variables, which receive a value related to a Modbus address. The data points are grouped into a data source environment, which can be connected using several communication protocols, such as Modbus IP or OPC. In the ScadaBR, a Modbus TCP server (defined as Modbus IP) was used, as viewed in Figure A3.

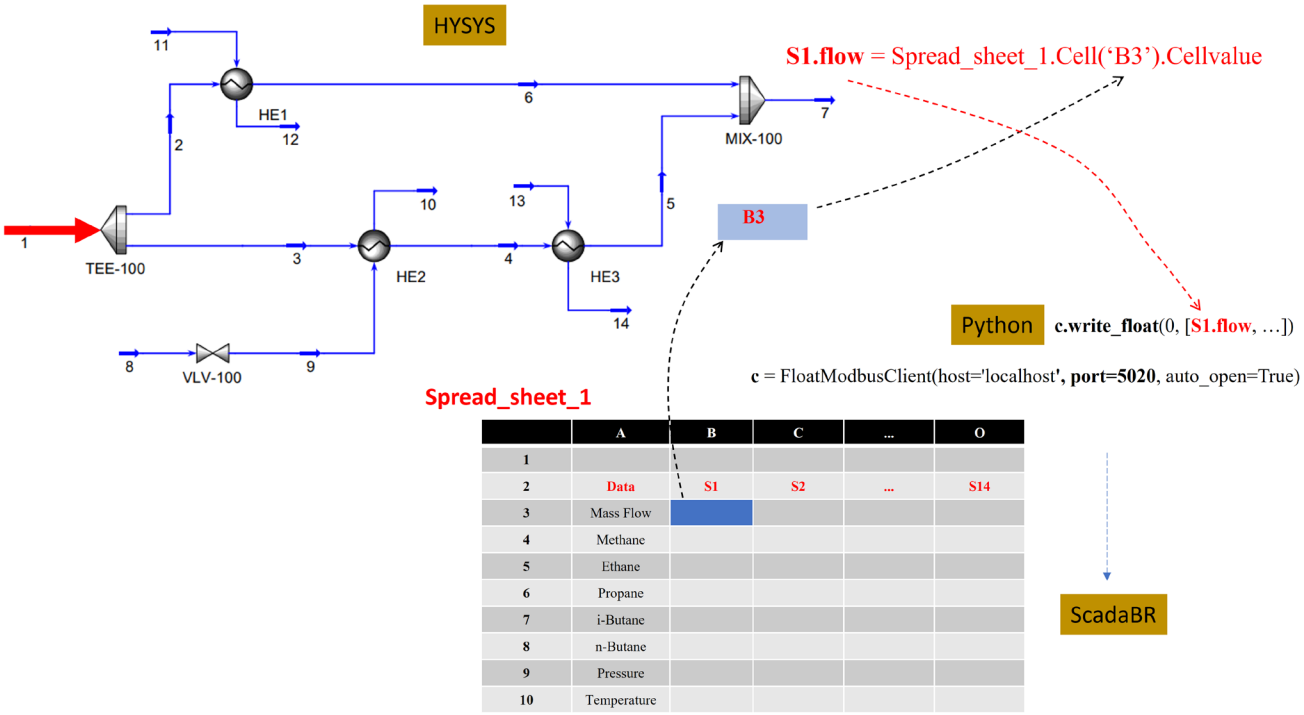

Figure A2. Communication between HYSYS–Python–ScadaBR (illustration of the Spread_sheet_1–"Simulation").

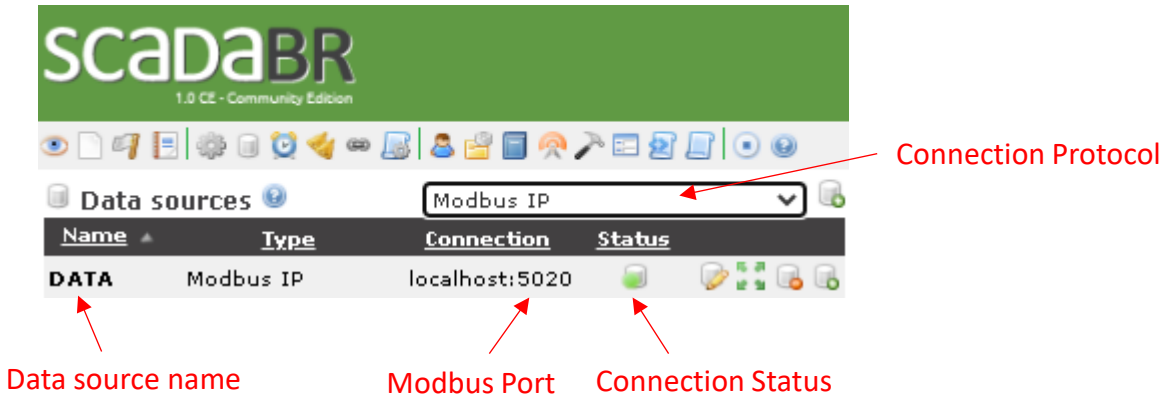

Figure A3. Adding a data source with Modbus IP server.

In "Modbus IP properties", Figure A4, it is possible to give a name to the data source, specify the sampling time, and the number of attempts to reconnect (in case of connection failure). In "Host", the communication address must be filled in, which in our case is

"localhost", and in "Port", the port used, which is 5020 for the first server and 5022 for the second Modbus server.

The position of variables stored in the Modbus TCP server in Python is defined by the vectors "c" and "d" written in the Python algorithm. Table A1 exemplifies the creation of datapoints for the parameters of the object S1.

Table A1. ModbusTCP adressess specification in ScadaBR.

Variable	ModbusTCP	Datapoint Offset
S1.flow	0	0
S1.ch4	1	2
S1.c2h6	2	4
S1.c3h8	3	6
S1.i_ch4h10	4	8
S1.n_ch4h10	5	10
S1.press	6	12
S1.temp	7	14

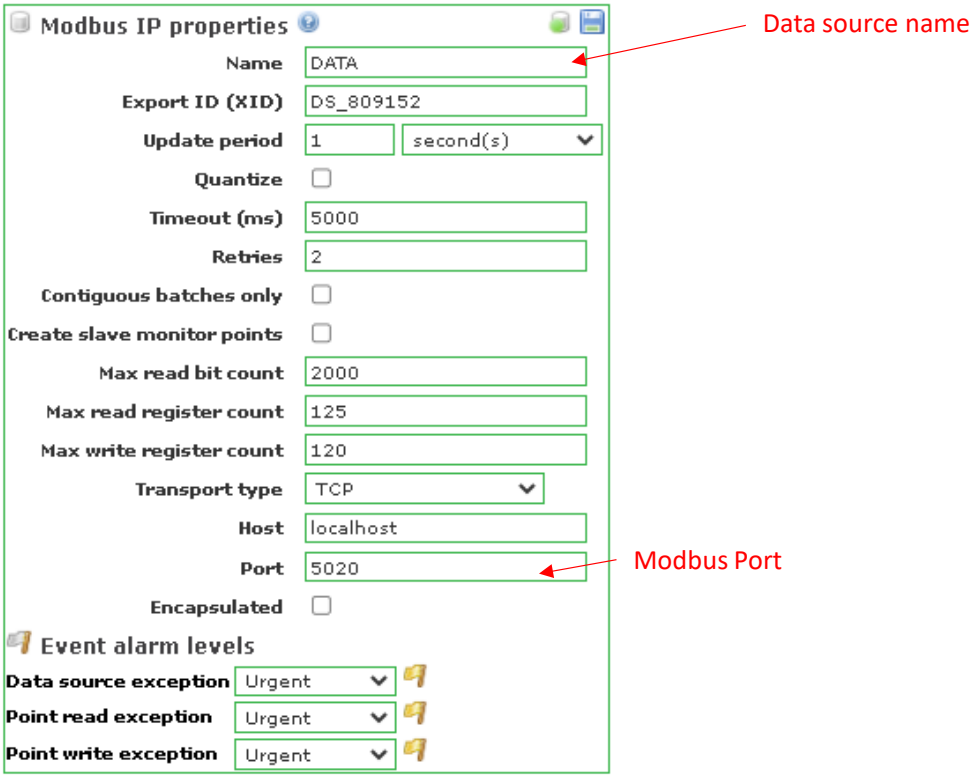

Figure A4. Modbus IP properties.

In Figure A5, the "Export ID (XID)" is set automatically, the "Slave Id" is not changed, remaining at the value of "1", the "Register Range" is set to "Recorder Holding", and the "Modbus Data Type" as "Float of 4 bytes". The option "Configurable" must be checked if the datapoint's value will be modified and exported to the Modbus server. The "Multiplier" and "Additive" factors are not changed, remaining at "1" and "0", respectively. These factors are used to modify the datapoint's value. Figure A6 illustrates all data point requirements created for a"S1" stream.

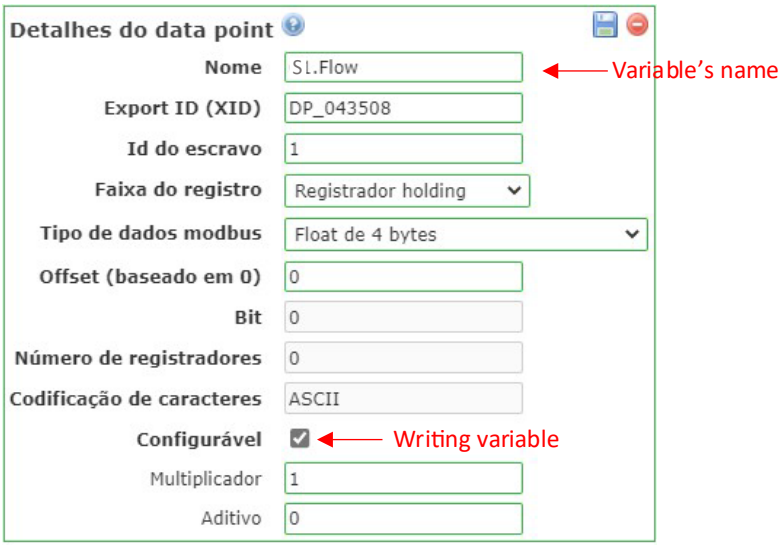

Figure A5. Data point details.

Figure A6. Enabled data points.

(b) Watch List

The watch list, Figure A7, is the easiest way to monitor the values of the imported process variables. To add a datapoint in the watch list just click on the green arrow next to it. This tool allows the user to visualize the value of the variable in real time, during the process simulation, and permits changing the values of the configurated datapoints. It is also possible to edit the watchlist by moving and removing data points. Additionally, in the "datapoint details" icon it is possible to visualize the statistics, history, and graphs regarding a specific variable.

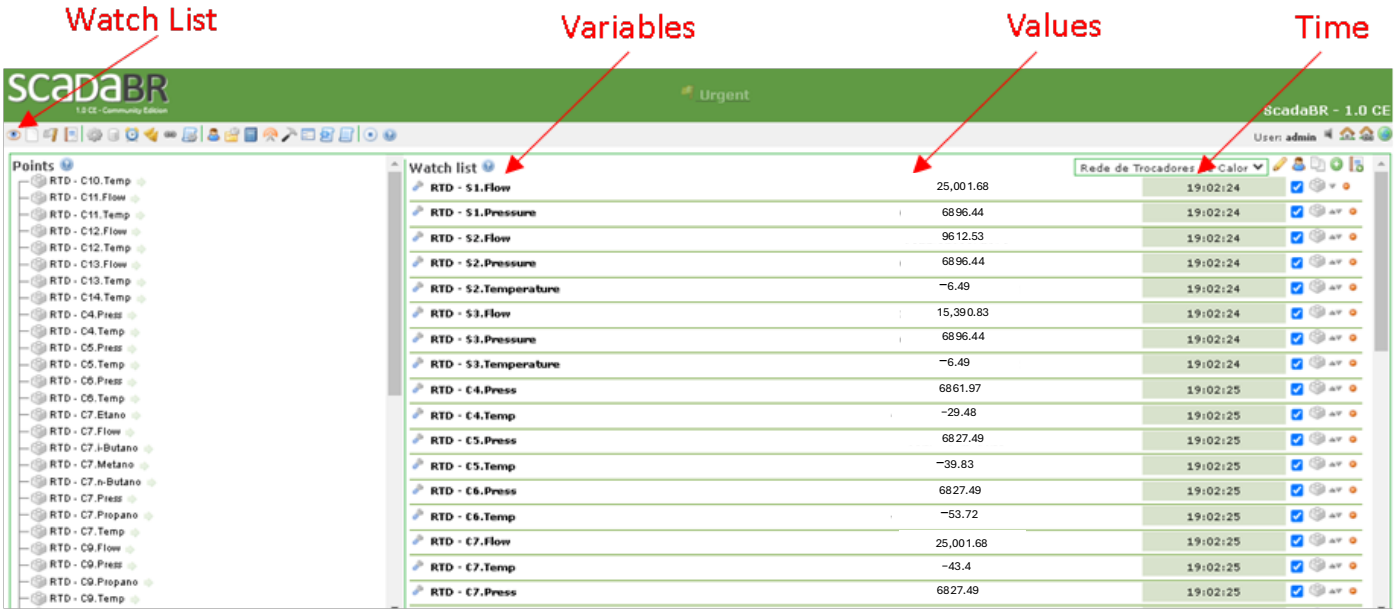

Figure A7. Watch list.

References

1. Chamorro-Atalaya, O.; Arce-Santillan, D.; Morales-Romero, G.; Trinidad-Loli, N.; Quispe-Andía, A.; Guzmán, E.; Lima-Perú, V.; León-Velarde, C. SCADA and Distributed Control System of a Chemical Products Dispatch Process. *Int. J. Adv. Comput. Sci. Appl.* **2021**, *12*. https://doi.org/10.14569/IJACSA.2021.0121288.
2. Kazala, R.; Straczynski, P. The Most Important Open Technologies for Design of Cost Efficient Automation Systems. *IFAC-PapersOnLine.* **2019**, *52*, 391–396. https://doi.org/10.1016/j.ifacol.2019.12.567
3. Rodríguez, F.; Guzmán, J.L.; Castilla, M.; Sánchez-Molina, J.A.; Berenguel, M. A Proposal for Teaching SCADA Systems Using Virtual Industrial Plants in Engineering Education. *IFAC-PapersOnLine* **2016**, *49*, 138–143.
4. Bellini, P.; Cenni, D.; Mitolo, N.; Nesi, P.; Pantaleo, G.; Soderi, M. High Level Control of Chemical Plant by Industry 4.0 Solutions. *J. Ind. Inf. Integr.* **2022**, *26*, 100276. https://doi.org/10.1016/j.jii.2021.100276.
5. Šenk, I.; Tegeltija, S.; Tarjan, L. Machine Learning in Modern SCADA Systems: Opportunities and Challenges. In Proceedings of the 2024 23rd International Symposium INFOTEH-JAHORINA (INFOTEH), Jahorina, Bosnia and Herzegovina, 20–22 March 2024; pp. 1–5.
6. Rehman, S.u.; Alhulayyil, H.; Alzahrani, T.; AlSagri, H.; Khalid, M.U.; Gruhn, V. Intrusion Detection System Framework for Cyber-Physical Systems. *Egypt. Inform. J.* **2025**, *30*, 100600. https://doi.org/10.1016/j.eij.2024.100600.
7. Lu, Q.; Shen, X.; Zhou, J.; Li, M. MBD-Enhanced Asset Administration Shell for Generic Production Line Design. *IEEE Trans. Syst. Man. Cybern. Syst*. **2024**, *54*, 5593–5605. https://doi.org/10.1109/TSMC.2024.3408296.
8. Zou, Z.; Cao, B.; Gui, X.; Chao, C. The Development of A Operator-Training Simulator Novel Type Chemical Process. Process Systems Engineering. *Comput. Aided Chem. Eng.* **2003**, *15*, 1447–1452.
9. Scholl, M.V.; Rocha, C.R. Embedded SCADA for Small Applications. *IFAC-PapersOnLine* **2016**, *49*, 246–253.
10. Bagal, K.N.; Kadu, C.B.; Parvat, B.J.; Vikhe, P.S. PLC Based Real Time Process Control Using SCADA and MATLAB. In Proceedings of the 2018 Fourth International Conference on Computing Communication Control and Automation (ICCUBEA), Pune, India, 16–18 August 2018; pp. 1–5.

11. Lakshmi Sangeetha, A.; Naveenkumar, B.; Balaji Ganesh, A.; Bharathi, N. Experimental Validation of PID Based Cascade Control System through SCADA-PLC-OPC and Internet Architectures. *Measurement* **2012**, *45*, 643–649. https://doi.org/10.1016/j.measurement.2012.01.005.
12. Dieu, B. Application of the SCADA System in Wastewater Treatment Plants. *ISA Trans.* **2001**, *40*, 267–281.
13. Morsi, I.; El-Din, L.M. SCADA System for Oil Refinery Control. *Measurement* **2014**, *47*, 5–13. https://doi.org/10.1016/j.measurement.2013.08.032.
14. Kovaliuk, D.O.; Huza, K.M.; Kovaliuk, O.O. Development of SCADA System Based on Web Technologies. *Int. J. Inf. Eng. Electron. Bus.* **2018**, *10*, 25–32. https://doi.org/10.5815/ijieeb.2018.02.04.
15. Novák, P.; Šindelář, R.; Mordinyi, R. Integration Framework for Simulations and SCADA Systems. *Simul. Model. Pract. Theory* **2014**, *47*, 121–140. https://doi.org/10.1016/j.simpat.2014.05.010.
16. Santos Bartolome, P.; Van Gerven, T. A Comparative Study on Aspen HYSYS Interconnection Methodologies. *Comput. Chem. Eng.* **2022**, 162, 107785. https://doi.org/10.1016/j.compchemeng.2022.107785.
17. Taqvi, S.A.; Tufa, L.D.; Muhadizir, S. Optimization and Dynamics of Distillation Column Using Aspen Plus®. *Procedia Eng.* **2016**, *148*, 978–984.
18. Franke, M.B. Mixed-Integer Optimization of Distillation Sequences with Aspen Plus: A Practical Approach. *Comput. Chem. Eng.* **2019**, *131*, 106583. https://doi.org/10.1016/j.compchemeng.2019.106583.
19. Lemos, I.d.S.; Santos, L. de S. Analysis of Multi-Objective Optimization Applied to the Simulation of a Natural Gas Separation Plant. *Gas Sci. Eng.* **2024**, *128*, 205360. https://doi.org/10.1016/J.JGSCE.2024.205360.
20. Siqueira, F.M.F.; Santos, L.d.S. Model-Optimization-Guided Neural Network (MOGNN) Applied to Chemical Processes. *Appl. Soft Comput.* **2024**, *167*, 112285. https://doi.org/10.1016/j.asoc.2024.112285.
21. De Souza, T.E.G. Modeling and Economic Optimization of an Industrial Site for Natural Gas Processing. *Digit. Chem. Eng.* **2022**, *6*, 100070.
22. Zhang, C.; Jun, K.; Gao, R.; Kwak, G.; Chang, S. Efficient Utilization of Associated Natural Gas in a Modular Gas-to-Liquids Process: Technical and Economic Analysis. *Fuel* **2016**, *176*, 32–39. https://doi.org/10.1016/j.fuel.2016.02.060.
23. Elyas, R. Dynamic Simulation for Process Control with Aspen HYSYS. In *Chemical Engineering Process Simulation*, 2nd ed.; Elsevier: Amsterdam, The Netherlands, 2022; pp. 309–341. https://doi.org/10.1016/B978-0-323-90168-0.00015-9.
24. Ghumare, S.; Kelly, T.E. Flare Minimisation via Dynamic Simulation. *Int. J. Environ. Pollut.* **2007**, *29*, 19.
25. Yamanee-Nolin, M.; Löfgren, A.; Andersson, N.; Nilsson, B.; Max-Hansen, M.; Pajalic, O. Single-Shooting Optimization of an Industrial Process through Co-Simulation of a Modularized Aspen Plus Dynamics Model. *Comput. Aided Chem. Eng.* **2019**, *46*, 721–726.
26. Asteasuain, M.; Tonelli, S.M.; Brandolin, A.; Bandoni, J.A. Dynamic Simulation and Optimisation of Tubular Polymerisation Reactors in GPROMS. *Comput. Chem. Eng.* **2001**, *25*, 509–515.
27. Khaled, M.S.; Imtiaz, S.; Ahmed, S.; Zendehboudi, S. Dynamic Simulation of Offshore Gas Processing Plant for Normal and Abnormal Operations. *Chem. Eng. Sci.* **2021**, *230*, 116159. https://doi.org/10.1016/j.ces.2020.116159.
28. Church, P.; Mueller, H.; Ryan, C.; Gogouvitis, S.V.; Goscinski, A.; Tari, Z. Migration of a SCADA System to IaaS Clouds—A Case Study. *J. Cloud Comput.* **2017**, *6*. https://doi.org/10.1186/s13677-017-0080-5.
29. ScadaBR. Available online: https://www.scadabr.com.br (accessed on 7 August 2025).
30. Miranda, T.d.C.R.D.; Santos, L.d.S. A Scada System Used for Monitoring Virtual Chemical Plants via Modbus Communication. *Internet Technol. Lett.* **2024**, *7*, e495. https://doi.org/10.1002/itl2.495.
31. Roman Savochenko OpenScada. Available online: http://oscada.org/ (accessed on 2 January 2026).
32. Luo, N.; Wang, X.; Van, F.; Ye, Z.C.; Qian, F. Integrated Simulation Platform of Chemical Processes Based on Virtual Reality and Dynamic Model. *Comput. Aided Chem. Eng.* **2015**, *37*, 581–586. https://doi.org/10.1016/B978-0-444-63578-5.50092-X.
33. Rapid SCADA. Available online: https://rapidscada.org/projects/ (accessed on 21 December 2025).
34. IndigoSCADA Available online: http://www.enscada.com/a7khg9/IndigoSCADA.html (accessed on 7 August 2025).
35. Edgar, T.F.; Himmelblau, D.M. *Optimization of Chemical Processes*; McGraw-Hill: New York, NY, USA, 2001.
36. Mohanty, S.; Saraf, D.N.; Kunzru, D. Modeling of a Hydrocracking Reactor. *Fuel Process. Technol.* **1991**, *29*, 1–17.
37. Barkhordari, A.; Fatemi, S.; Daneshpayeh, M.; Zamani, H. Kinetic Modeling of Industrial VGO Hydrocracking in a Life Term of Catalyst. *Int. J. Chem. React. Eng.* **2010**, *8*, 1–26.
38. Botelho, D. Modelagem e Simulação do Processo de Hidrocraqueamento Catalítico de Frações Pesadas de Petróleo. Master's Thesis, Universidade Federal Fluminense, Niterói, Brazil, 2017.

39. Martínez, J.; Ancheyta, J. Kinetic Model for Hydrocracking of Heavy Oil in a CSTR Involving Short Term Catalyst Deactivation. *Fuel* **2012**, *100*, 193–199.
40. Luque-Rodríguez, S.; Vega-Granda, A.B. *Simulación y Optimización Avanzadas en la Industria Química y de Procesos: HYSYS*; Universidad de Oviedo: Oviedo, Spain, 2014; ISBN 9780874216561.
41. Smith, J.M.; Van Ness, H.C.; Abbott, M.M. *Introduction to Chemical Engineering Thermodynamics*, 7th ed.; McGraw-Hill, Inc.: New York, NY, USA, 2005.
42. Seborg, D.E.; Edgar, T.F.; Mellichamp, D.A.; Doyle, F.J. *Process Dynamics and Control*; Wiley: Hoboken, NJ, USA, 2011.
43. Nelder, J.A.; Mead, R. A Simplex Method for Function Minimization. *Comput. J.* **1965**, *7*, 308–313. https://doi.org/10.1093/comjnl/7.4.308.
44. Antunes, L.S. Simulação e Otimização do Processo de Hidrocraqueamento Catalítico de Frações Pesadas de Petróleo. Bachelor's Thesis, Universidade Federal Fluminense, Niterói, Brazil, 2022.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.